

THE ELLIPSOID METHOD AND SEPARATION ORACLES: A PRACTICAL SURVEY*

WAI-SHING LUK[†]

Abstract. The ellipsoid method is a powerful optimization technique that offers distinct advantages over interior-point methods, as it does not require the evaluation of all constraint functions. This makes it a natural choice for convex problems with numerous or even infinite constraints. The method employs an ellipsoid as a search space and relies on a separation oracle to provide cutting planes for updating it. It is worth noting that the significance of the separation oracle is often overlooked. This article evaluates the utility of the ellipsoid method in three distinct applications: robust convex optimization, semidefinite programming, and parametric network optimization. The effectiveness of separation oracles is assessed for each application. Furthermore, this article addresses the implementation issues associated with the ellipsoid method, including the utilization of parallel cuts for updating the ellipsoid. In certain cases, the use of parallel cuts has been observed to reduce computation time, as evidenced in the context of FIR filter design. The article also considers discrete optimization, demonstrating how the ellipsoid method can be applied to problems involving quantized discrete design variables. The additional effort in oracle implementation is limited to locating the nearest discrete solutions. The advantages of the method are nevertheless bounded: it cannot exploit sparsity in the problem data, and its iteration budget grows as n^2 , so it is most effective when the number of design variables is moderate; its practical performance further depends on compiled execution and on careful floating-point safeguards.

1. Introduction. The reputation of the ellipsoid method is negatively impacted by its perceived slower performance in solving large-scale convex problems when compared to the interior-point method. This perception is, however, an unfair one. In contrast to the interior-point method, the ellipsoid method does not require the explicit evaluation of all constraint functions. Instead, the method employs an ellipsoid as a search space and requires only a separation oracle that furnishes a *cutting plane* (§2). This method is particularly well-suited to problems that involve a moderate number of design variables but have a large number of constraints, or even an infinite number of constraints. The method cannot exploit sparsity in the data, but the separation oracle can exploit structural properties of the problem.

Despite decades of research into the ellipsoid method [3], the importance of the separation oracle is often overlooked. This article examines three specific applications: robust convex optimization, network optimization, and semidefinite programming. The effectiveness of separation oracles is assessed for each application.

Beyond surveying oracle techniques, the article contributes original implementation methodology. These contributions include a split representation of the ellipsoid that lowers the per-iteration cost (§4.1.1), parallel cuts for two-sided constraints (§4.3), a numerically stable factorized update and a floating-point stall guard (§4.1.2, §2.2.1), the embedding of quantized constraints inside the oracle (§4.8), and a cross-language benchmark against an interior-point solver (§4.6). The treatment is therefore both a review of how separation oracles are constructed and a practitioner’s account of how the resulting method is made to work. §2 establishes the cutting-plane framework, §3 surveys the oracles for robust, network, and matrix-inequality problems, and §4 develops the ellipsoid update together with its numerical safeguards.

*This work was supported by ...

[†]Fudan University

Robust optimization incorporates parameter uncertainties into the optimization problem by analyzing the worst-case scenario. The objective is to find a solution that is both reliable and performs optimally under a range of possible parameter values within a specified set of uncertainties. A robust counterpart of a convex problem preserves its convexity, despite the number of constraints growing to infinity. This renders the ellipsoid method an excellent choice for addressing such problems. For further details, see §3.1.

Furthermore, an illustration is provided of a network optimization scenario in which the ellipsoid method can be utilized. The separation oracle involves the construction of a cutting plane by finding a negative cycle within a network graph. There are algorithms available for finding negative cycles that leverage network locality and other properties, resulting in an efficient implementation of oracles. For a more detailed discussion, please refer to §3.2.

Finally, §3.3 addresses matrix inequalities. Recall that Cholesky or LDL^T decomposition allows the positive definiteness of a symmetric matrix to be checked efficiently. If a symmetric matrix A , with dimensions of $m \times m$, encounters a non-positive diagonal entry during decomposition, and the process is terminated at row p , then A cannot be positive definite. In such cases, a witness vector v can be constructed to certify that A is not positive definite. The row-based decomposition and lazy evaluation technique enable the cutting plane to be constructed in $O(p^3)$, thus allowing its use in efficient oracle implementations.

The implementation of the ellipsoid method is discussed in greater detail in §4. In essence, the method generates a sequence of ellipsoids whose volume is uniformly decreased at each step, thereby ensuring a systematic and consistent approach to volume reduction. The ellipsoid is typically represented as follows:

$$\{x \mid (x - x_c)P^{-1}(x - x_c) \leq 1\},$$

where $x_c \in \mathbb{R}^n$ is the center of the ellipsoid. The matrix $P \in \mathbb{R}^{n \times n}$ is symmetric positive definite. In each iteration, the ellipsoid method performs updates to both x_c and P . Although updating ellipsoids is a relatively straightforward process that has been implemented for decades, we show that the cost can be reduced by an additional n^2 floating-point operations by splitting the matrix P into two parts, namely κ and Q . This yields the following form:

$$\{x \mid (x - x_c)Q^{-1}(x - x_c) \leq \kappa\}.$$

Moreover, §4.3 addresses the utilization of parallel cuts. Some researchers have suggested that this technique does not result in significant improvements. Nevertheless, our findings indicate that in scenarios where specific constraints are subject to narrow upper and lower bounds, such as in the context of FIR filter designs, the incorporation of parallel cuts can markedly reduce the runtime. Furthermore, we demonstrate that when the ellipsoid method is implemented with precision, any update, whether it employs a single cut or a parallel cut, requires at most one square root.

In many practical engineering problems, some design variables may be constrained to discrete forms. Since the cutting-plane method requires only a separation oracle, it can also be used for discrete problems. The only additional effort in the oracle implementation is finding the nearest discrete solution.

2. Cutting-Plane Method Revisited.

2.1. Convex Feasibility Problem. Let $\mathcal{K}$ be a compact and convex subset of $\mathbb{R}^n$. Consider the following feasibility problem:

1. Find a point $x^* \in \mathbb{R}^n$ in $\mathcal{K}$, or
2. Determine if $\mathcal{K}$ is empty, i.e. if it has no feasible solution.

A separation oracle, also known as a cutting-plane oracle, is the mechanism by which a cutting-plane method accesses a convex set. When a separation oracle, denoted by Ω , is queried at a given point $x_0 \in \mathbb{R}^n$, it can produce one of the following outputs:

1. It asserts that x_0 belongs to $\mathcal{K}$, or
2. Returns a hyperplane separating the point x_0 from the set $\mathcal{K}$:

$$g^\top(x - x_0) + \beta \leq 0, g \neq 0, \forall x \in \mathcal{K}.$$

The pair (g, β) is called a *cutting plane* because it eliminates the half-space defined by the equation $\{x \mid g^\top(x - x_0) + \beta > 0\}$ from the search space. The following observations are made:

- If $\beta = 0$, indicating that x_0 is on the boundary of the half-space, the cutting plane is called a *central cut*.
- If $\beta > 0$, indicating that x_0 is inside the half-space, the cutting plane is called a *deep cut*.
- If $\beta < 0$, indicating that x_0 is outside the half-space, the cutting plane is called a *shallow cut*.

The convex set $\mathcal{K}$ is typically defined by a set of inequalities $f_j(x) \leq 0$ or $f_j(x) < 0$ for $j = 1, \dots, m$, where $f_j(x)$ represents a convex function. The vector $g \in \partial f(x_0)$ is defined as the *subgradient* of a convex function f at the point x_0 if $f(z) \geq f(x_0) + g^\top(z - x_0)$. Thus, the cut (g, β) can be expressed as $(\partial f(x_0), f(x_0))$. Note that if $f(x)$ is differentiable, then we can simply take $\partial f(x_0) = \nabla f(x_0)$.

The cutting-plane method consists of two main elements: a separation oracle, denoted by Ω , and a search space, denoted by $\mathcal{S}$, which is initially chosen large enough to encompass $\mathcal{K}$. For example,

- Polyhedron $\mathcal{P} = \{z \mid Cz \preceq d\}$.
- Ellipsoid $\mathcal{E} = \{z \mid (z - x_c)^\top P^{-1}(z - x_c) \leq 1\}$.
- Interval $\mathcal{I} = [l, u]$ (for one-dimensional problem).

Let us denote the center of the current set, denoted by $\mathcal{S}$, as x_c . The following is a basic outline of the methodology underlying the cutting-plane method:

1. **Initialization:** The initial stage of the method involves defining a search space $\mathcal{S}$ that is guaranteed to contain a point x^* .

Algorithm 1 Cutting-plane feasibility

Require: oracle Ω , initial ellipsoid $\mathcal{E} \supseteq \mathcal{K}$, tolerance ϵ

Ensure: a point $x^* \in \mathcal{K}$, or “infeasible”

```
1: repeat  
2:    $x_c \leftarrow \text{center}(\mathcal{E})$   
3:    $(\text{status}, g, \beta) \leftarrow \Omega(x_c)$   
4:   if  $\text{status} = \text{feasible}$  then  
5:     return  $x_c$   
6:   end if  
7:    $\mathcal{E} \leftarrow \text{UPDATE}(\mathcal{E}, g, \beta)$   
8: until  $\text{vol}(\mathcal{E}) < \epsilon$   
9: return “infeasible”
```

2. **Iteration:** In each iteration, the separation oracle is queried at the center x_c . If x_c is in $\mathcal{K}$, then the iteration is terminated.

3. **Update:** The smaller search space, denoted by $\mathcal{S}^+$, is computed and contains the half-space from step 2.

4. **Repeat:** Repeat steps 2 and 3 until $\mathcal{S}$ is either empty or sufficiently small.

2.2. From Feasibility to Optimization. Let us now turn our attention to the following consideration:

$$\begin{aligned} & \text{minimize} && f_0(x), \\ & \text{subject to} && x \in \mathcal{K}. \end{aligned}$$

The objective $f_0(x)$ may be convex or quasiconvex. The aforementioned optimization problem is treated as a feasibility problem with an additional constraint, namely that $f_0(x) \leq \gamma$, where $\gamma \in \mathbb{R}$ is called the best-so-far value of $f_0(x)$. Accordingly, the problem can be reformulated as follows:

$$\begin{aligned} & \text{minimize} && \gamma, \\ & \text{subject to} && \Phi(x, \gamma) \leq 0, \\ & && x \in \mathcal{K}, \end{aligned}$$

where $\Phi(x, \gamma) \leq 0$ is the γ -sublevel set of $f_0(x)$ when $f_0(x)$ is quasiconvex. For every x , $\Phi(x, \gamma)$ is a non-increasing function of γ , i.e., $\Phi(x, \gamma') \leq \Phi(x, \gamma)$ whenever $\gamma' \geq \gamma$. Let $\mathcal{K}_\gamma$ denote the new constraint set.

One straightforward approach to solving the optimization problem is to perform a binary search on γ and solve the corresponding feasibility problems at each value of γ . An alternative approach is to update the current best estimate of γ whenever a feasible solution x_0 is found such that $\Phi(x_0, \gamma) = 0$.

The following is a basic outline of the operational procedure of the cutting-plane method (optim):

1. **Initialization:** The initial stage of the process entails defining a search space $\mathcal{S}$ that is guaranteed to contain a solution, x^* .

Algorithm 2 Cutting-plane optimization

Require: oracle Ω , initial ellipsoid $\mathcal{E} \supseteq \mathcal{K}_{\gamma_0}$

Ensure: best point x^* and value γ^*

```
1:  $\gamma \leftarrow +\infty$ 
2: repeat
3:    $x_c \leftarrow \text{center}(\mathcal{E})$ 
4:    $(g, \beta, t) \leftarrow \Omega(x_c, \gamma)$ 
5:   if  $t < \gamma$  then
6:      $\gamma \leftarrow t$ ;  $x^* \leftarrow x_c$ ;  $\mathcal{E} \leftarrow \text{CENTRALCUT}(\mathcal{E}, g, \beta)$ 
7:   else
8:      $\mathcal{E} \leftarrow \text{DEEPCUT}(\mathcal{E}, g, \beta)$ 
9:   end if
10: until  $\text{vol}(\mathcal{E}) < \epsilon$ 
11: return  $x^*, \gamma$ 
```

2. **Iteration:** In each iteration, the separation oracle is queried at the point x_c . A subgradient of the function at x_c must then be computed. This results in the generation of a half-space that is guaranteed to contain x^* .
3. If $x_c \in \mathcal{K}_\gamma$, update γ such that $\Phi(x_c, \gamma) = 0$.
4. **Update:** The smaller $\mathcal{S}^+$ that contains the half-space from step 2 is computed.
5. **Repeat:** Repeat steps 2 to 4 until $\mathcal{S}$ is either empty or sufficiently small.

Generic cutting-plane method (Optim)

- **Given** an initial $\mathcal{S}$ known to contain $\mathcal{K}_\gamma$.
- **Repeat**
 1. Select a point x_0 in $\mathcal{S}$
 2. Query the separation oracle at x_0
 3. **If** $x_0 \in \mathcal{K}_\gamma$, update γ so that $\Phi(x_0, \gamma) = 0$.
 4. Update $\mathcal{S}$ to a smaller set that covers:

$$\mathcal{S}^+ = \mathcal{S} \cap \{z \mid g^\top(z - x_0) + \beta \leq 0\}$$

5. **If** $\mathcal{S}^+ = \emptyset$ or it is small enough, exit.

We assume that the oracle takes responsibility for this update.

2.2.1. Termination Criteria and the Stall Guard. The optimization form of the cutting-plane method is often driven by a binary search on the best-so-far value γ , and the search terminates when the width of the current bracket falls below a prescribed tolerance. A common implementation compares this width, which scales with the magnitude of the bracket, against an absolute constant. Such a test can never be satisfied: once the bracket has collapsed to the resolution of floating-point arithmetic, the midpoint is the only representable point it contains, and the width stalls at approximately the unit in the last place of the bracket end points. If the tolerance is smaller than this resolution, the loop stops shrinking but continues to iterate until an unrelated iteration cap is reached, pinning the iteration counter and wasting inner subproblem solves on brackets that cannot be refined further.

The remedy is a stall guard rather than a tighter tolerance. Before accepting the

midpoint, the driver checks whether it is strictly inside the bracket,

$$\gamma = \text{lower} + \tau, \quad \text{lower} < \gamma < \text{upper},$$

and terminates when this test fails, where τ is the half-width. When the guard fires, no further refinement is representable, so terminating cannot alter the answer; the returned value is identical to the one a converged tolerance test would have produced. This turns an implementation-dependent iteration count into one governed by the information content of the floating-point format. The same caution applies to the volume of the ellipsoid: an absolute stopping threshold on a quantity that scales with the problem data should always be paired with a stall detection test.

2.3. Example: Profit Maximization. This example is taken from the article “Robust Optimization for Profit Maximization” by Aliabadi (2013) [2]. We will consider the following *short-run* profit maximization problem:

$$(2.1) \quad \begin{array}{ll} \text{maximize} & p(Ax_1^\alpha x_2^\beta) - v_1 x_1 - v_2 x_2, \\ \text{subject to} & x_1 \leq k, \\ & x_1 > 0, x_2 > 0, \end{array}$$

where the variable A represents the scale of production, while the variables α and β denote output elasticities. The x_i and v_i terms refer to the quantity and price of the i -th input, respectively. The term $Ax_1^\alpha x_2^\beta$ is the Cobb-Douglas production function, which is a widely accepted model used to represent the relationship between inputs and outputs in production. The quantity of x_1 is constrained by the constant k . The formulation above is not convex. To begin, we will reformulate the problem as follows:

$$\begin{array}{ll} \text{maximize} & \gamma, \\ \text{subject to} & \gamma + v_1 x_1 + v_2 x_2 \leq pAx_1^\alpha x_2^\beta, \\ & x_1 \leq k, \\ & x_1 > 0, x_2 > 0. \end{array}$$

By means of a change of variables, the following convex form of Eq. 2.1 can be obtained:

$$\begin{array}{ll} \text{maximize} & \gamma, \\ \text{subject to} & \log(\gamma + v_1 e^{y_1} + v_2 e^{y_2}) - (\alpha y_1 + \beta y_2) \leq \log(pA), \\ & y_1 \leq \log k, \end{array}$$

{#eq:profit-in-cvx-form} where $y_1 = \log x_1$ and $y_2 = \log x_2$.

Some readers may recognize that the problem can also be written in a geometric program by introducing one additional variable [2].

3. Oracle Constructions for Three Applications.

- Robust convex optimization
 - oracle technique: affine arithmetic
- Parametric network potential problem

- oracle technique: negative cycle detection
- Semidefinite programming
 - oracle technique: Cholesky decomposition

3.1. Robust Convex Optimization. Robust optimization addresses parameter uncertainty by optimizing for the worst case, which yields solutions that stay reliable across an entire uncertainty set. This study treats profit maximization as a robust geometric program under interval uncertainty. The authors work with the Cobb-Douglas production function and approximate the robust counterpart by piecewise convex linear functions, expressed as a geometric program; an illustrative example follows.

Here the model parameters are represented as intervals, and the authors provide upper and lower piecewise convex linear approximations of the robust counterpart, which can be solved by interior-point methods.

For the purposes of this discussion, we will consider:

$$(3.1) \quad \begin{array}{ll} \text{minimize} & \sup_{q \in \mathcal{Q}} f_0(x, q), \\ \text{subject to} & f_j(x, q) \leq 0, \forall q \in \mathcal{Q}, j = 1, 2, \dots, m, \end{array}$$

where q denotes the vector of uncertain parameters. The issue can be rephrased as follows:

$$\begin{array}{ll} \text{minimize} & \gamma, \\ \text{subject to} & f_0(x, q) \leq \gamma, \\ & f_j(x, q) \leq 0, \forall q \in \mathcal{Q}, j = 1, 2, \dots, m. \end{array}$$

3.1.1. Algorithm. The oracle is responsible for determining the following:

- If $f_j(x_0, q) > 0$ for some j and $q = q_0$, then the cut (g, β) is equal to $(\partial f_j(x_0, q_0), f_j(x_0, q_0))$.
- If $f_0(x_0, q) \geq \gamma$ for some $q = q_0$, then the cut (g, β) is equal to $(\partial f_0(x_0, q_0), f_0(x_0, q_0) - \gamma)$.
- Otherwise, if x_0 is feasible, then
 - Let $q_{\max} = \arg \max_{q \in \mathcal{Q}} f_0(x_0, q)$.
 - $\gamma := f_0(x_0, q_{\max})$.
 - The cut (g, β) is equal to $(\partial f_0(x_0, q_{\max}), 0)$.

3.1.2. Example: Robust Profit Maximization. Let us revisit the profit maximization problem in §2.3. The model parameters are subject to uncertainty over a given interval. Let us now consider the case in which the parameters $\alpha, \beta, p, v_1, v_2$, and k are subject to interval uncertainties, as outlined in [2]:

$$\begin{array}{rclcl} \alpha - \varepsilon_1 & \leq & \hat{\alpha} & \leq & \alpha + \varepsilon_1 \\ \beta - \varepsilon_2 & \leq & \hat{\beta} & \leq & \beta + \varepsilon_2 \\ p - \varepsilon_3 & \leq & \hat{p} & \leq & p + \varepsilon_3 \\ v_1 - \varepsilon_4 & \leq & \hat{v}_1 & \leq & v_1 + \varepsilon_4 \\ v_2 - \varepsilon_5 & \leq & \hat{v}_2 & \leq & v_2 + \varepsilon_5 \\ k - \varepsilon_6 & \leq & \hat{k} & \leq & k + \varepsilon_6 \end{array}$$

7

The problem formulation of the robust counterpart considering the worst-case scenario is:

$$\begin{aligned} \max \quad & \gamma \\ \text{subject to} \quad & \log(\gamma + \hat{v}_1 e^{y_1} + \hat{v}_2 e^{y_2}) - (\hat{\alpha} y_1 + \hat{\beta} y_2) \leq \log(\hat{p} A) \\ & y_1 \leq \log \hat{k}. \end{aligned}$$

The piecewise convex linear approximation of [2] makes the robust counterpart tractable for interior-point algorithms, but it requires substantial programming effort and yields inherently imprecise solutions. Both drawbacks are avoided by the cutting-plane method. In this simple example the worst-case scenario occurs when:

- $\hat{p} = p - \varepsilon_3$, $\hat{k} = k - \varepsilon_6$
- $\hat{v}_1 = v_1 + \varepsilon_4$, $\hat{v}_2 = v_2 + \varepsilon_5$,
- if $y_1 > 0$, $\hat{\alpha} = \alpha - \varepsilon_1$; otherwise $\hat{\alpha} = \alpha + \varepsilon_1$
- if $y_2 > 0$, $\hat{\beta} = \beta - \varepsilon_2$; otherwise $\hat{\beta} = \beta + \varepsilon_2$

It is even possible to reuse the original oracle to compose the robust counterpart.

It should be noted that the “argmax” may be non-convex, which may make it challenging to solve. For more complex problems, one potential approach is to utilize affine arithmetic as a computational aid [19].

3.1.3. Affine Arithmetic and the Worst-Case Oracle. The robust oracle for Eq. 3.1 must repeatedly evaluate the worst-case value $\sup_{q \in \mathcal{Q}} f_j(x, q)$ and differentiate it with respect to x . When $\mathcal{Q}$ is a box, that is, the Cartesian product of finitely many intervals, the naive strategy evaluates f_j at every vertex of the box. The number of vertices is exponential in the number of uncertain parameters, so this scenario-enumeration oracle is affordable only for a handful of uncertainties and is poorly suited to the cutting-plane method, which may require many oracle calls along the trajectory.

Affine arithmetic replaces enumeration by a first-order propagation of uncertainty. Each uncertain parameter is written once in the form

$$\hat{q} = q_0 + \sum_{k=1}^K q_k \epsilon_k, \quad \epsilon_k \in [-1, 1],$$

where $q_0, \dots, q_K$ are known constants and the symbols ϵ_k are independent. Every intermediate quantity produced while evaluating $f_j(x, \cdot)$ is kept in the same affine representation. Addition and multiplication by constants are exact, whereas a general nonlinear operation is replaced by its affineization together with an error term that bounds the residual curvature. The essential point is that, for a fixed x , the propagated coefficients are obtained at a cost proportional to the size of the expression and to the number of symbols K , independently of the number of vertices of $\mathcal{Q}$.

Because the resulting form is affine in the symbols, its range over the box is available in closed form. Writing the affine approximation of the j -th constraint as

$$f_j(x, q) \approx c_0(x) + \sum_{k=1}^K c_k(x) \epsilon_k,$$

the extreme values of f_j over $\mathcal{Q}$ are

$$\underline{f}_j(x) = c_0(x) - \sum_{k=1}^K |c_k(x)|, \quad \bar{f}_j(x) = c_0(x) + \sum_{k=1}^K |c_k(x)|.$$

The upper end point $\bar{f}_j(x)$ is the worst-case value predicted by the affine model, and the sign pattern of the coefficients identifies the scenario $\epsilon_k = \text{sign}(c_k(x))$ that attains it. The oracle then evaluates the true functions at that scenario and issues the cut $(g, \beta) = (\partial f_j(x_0, q_0), f_j(x_0, q_0))$ prescribed in the algorithm above. If $\bar{f}_j(x) \leq 0$ for every j at an accepted point, the design is certified feasible for all $q \in \mathcal{Q}$.

Two properties make affine arithmetic attractive in this setting. First, it subsumes ordinary interval arithmetic: by retaining the linear dependence on each symbol, it removes much of the overestimation caused by the dependency problem, in which the same variable occurring twice would otherwise be treated as two independent quantities. Second, its cost grows polynomially rather than exponentially in the number of uncertain parameters, which is precisely the regime in which the robust counterpart would otherwise be intractable. The approach was proposed for robust geometric programming in [19].

3.2. Multi-parameter Network Problems. In the context of network theory, a directed graph, denoted by $G = (V, E)$, represents a network. Let us consider the following:

$$\begin{array}{ll} \text{minimize} & \gamma, \\ \text{subject to} & u_i - u_j \leq h_{ij}(x, \gamma), \quad \forall (i, j) \in E, \\ \text{variables} & x, u, \end{array}$$

where $h_{ij}(x, \gamma)$ is the weight function of edge (i, j) .

It is assumed that the network is of a considerable size, but that the number of parameters is relatively limited. The problem has a feasible solution if and only if G contains no negative cycles. Let $\mathcal{C}$ be a set of all cycles of G . The problem can be formulated as follows:

$$\begin{array}{ll} \text{minimize} & \gamma, \\ \text{subject to} & W_k(x, \gamma) \geq 0, \quad \forall C_k \in \mathcal{C}, \\ \text{variables} & x, \end{array}$$

where C_k is a cycle of G :

$$W_k(x, \gamma) = \sum_{(i,j) \in C_k} h_{ij}(x, \gamma).$$

The minimum cycle ratio (MCR) problem is a fundamental problem in the analysis of directed graphs. Given a directed graph, the MCR problem seeks to find the cycle with the minimum ratio of the sum of the edge weights to the number of edges in the cycle. In other words, the MCR problem seeks to find the “tightest” cycle in the

graph, where the tightness of a cycle is measured by the ratio of the total weight of the cycle to its length.

The MCR problem has numerous applications in the analysis of discrete event systems, including digital circuits and communication networks. It is closely related to other problems in graph theory, such as the shortest path problem and the maximum flow problem. Consequently, efficient algorithms for solving the MCR problem are of great practical importance.

3.2.1. Negative Cycle Detection Algorithm. The most time-consuming part of the proposed method is the negative cycle detection, which underscores the importance of selecting an appropriate negative cycle detection algorithm. There are numerous methods for detecting negative cycles in weighted graphs [6]. Tarjan’s algorithm [29] is one of the fastest in practice and is widely regarded as a benchmark for this purpose [6, 8].

Howard’s method employs policy iteration to find the minimum cycle ratio (MCR) of a directed graph: it maintains a set of candidate cycles and repeatedly updates the cycle of minimum ratio until convergence.

The separation oracle is only required to determine:

- If a negative cycle C_k exists under x_0 , then the cut is $(g, \beta) = (-\partial W_k(x_0), -W_k(x_0))$.
- If $f_0(x_0) \geq \gamma$, then the cut $(g, \beta) = (\partial f_0(x_0), f_0(x_0) - \gamma)$.
- Otherwise, if x_0 is feasible, then
 - $\gamma := f_0(x_0)$.
 - the cut is $(g, \beta) = (\partial f_0(x_0), 0)$.

3.2.2. Example: Optimal matrix scalings under the min-max-ratio criterion. The following example is taken from [22]. As stated by [22], optimal matrix scaling has a number of practical applications. One such application is in the field of linear programming, where groups of constraints and groups of variables may represent the same physical commodity for which common measurement units are employed. Another application is in telecommunications, where matrix scaling helps optimize signal transmission. Matrix scaling has also been used in approximation theory, to approximate functions of several variables by sums of functions of fewer variables, and in Gaussian elimination, to improve numerical stability when solving linear systems.

Let us consider a matrix $A \in \mathbb{R}^{N \times N}$. A *symmetric scaling* of A is defined as a matrix B of the form UAU^{-1} , where U is a nonnegative diagonal matrix of the same dimension. In accordance with the *min-max criterion*, the objective is to minimize the largest absolute value of B ’s elements [22, (Program 3)]:

$$\begin{aligned} & \text{minimize} && \pi \\ & \text{subject to} && 1 \leq u_i |a_{ij}| u_j^{-1} \leq \Pi, \quad \forall a_{ij} \neq 0, \\ & && \pi, u_1 \cdot u_N \text{ positive.} \end{aligned}$$

The authors demonstrate that the problem of determining the optimal symmetric scalings under the min-max criterion can be transformed into a single-parameter network optimization problem. This can be solved efficiently using parametric network algorithms.

Another possible criterion is to minimize the ratio of the largest absolute value of the element B to the smallest. One rationale for employing this criterion is that high ratios impede the efficacy of the simplex method. With this *min-max-ratio* criterion, the symmetric scaling problem can be formulated as [22, (Program 8)]:

$$\begin{aligned} & \text{minimize} && \pi/\psi \\ & \text{subject to} && \psi \leq u_i |a_{ij}| u_j^{-1} \leq \Pi, \forall a_{ij} \neq 0, \\ & && \pi, \psi, u_1 \cdot u_N \text{ positive.} \end{aligned}$$

Let $k' = \log |k|$. Taking logarithms of the variables transforms the program above into a two-parameter network problem:

$$\begin{aligned} & \text{minimize} && \pi' - \psi' \\ & \text{subject to} && u'_i - u'_j \leq \pi' - a'_{ij}, \forall a_{ij} \neq 0, \\ & && u'_j - u'_i \leq a'_{ij} - \psi', \forall a_{ij} \neq 0, \\ & \text{variables} && \pi', \psi', u'. \end{aligned}$$

where $x = (\pi', \psi')^\top$. The authors of [22] assert that they have developed an algorithm for solving multi-parameter problems. Nevertheless, we were unable to identify any follow-up publications that corroborate this assertion. It is noteworthy that the cutting-plane method readily extends the single-parameter network algorithm to accommodate multi-parameter problems.

In this application, the function $h_{ij}(x)$ is defined as follows:

$$h_{ij}(x) = \begin{cases} -\pi' + a'_{ij}, & \forall a_{ij} \neq 0, \\ \psi' - a'_{ji}, & \forall a_{ji} \neq 0, \end{cases}$$

Fast algorithms for finding a negative cycle can be found in [7, 9]. Further applications to clock skew scheduling can be found in [33].

3.3. Problems Involving Matrix Inequalities. Consider the following problem:

$$\begin{aligned} & \text{find} && x, \\ & \text{subject to} && F(x) \succeq 0, \end{aligned}$$

where F is a matrix-valued function and $F(x) \succeq 0$ means that $F(x)$ is positive semidefinite. It should be recalled that a matrix A is positive semidefinite if and only if $v^\top A v \geq 0$ for all $v \in \mathbb{R}^N$. The problem can be transformed into the following form:

$$\begin{aligned} & \text{find} && x, \\ & \text{subject to} && v^\top F(x) v \geq 0, \forall v \in \mathbb{R}^N. \end{aligned}$$

Consider the case where $v^\top F(x)v$ is concave for all $v \in \mathbb{R}^N$ with respect to x , then the problem above is convex. The problem can be reduced to *semidefinite programming* if the function $F(x)$ is linear with respect to x , i.e., $F(x) = F_0 + x_1 F_1 + \cdots + x_n F_n$, where $F_0, F_1, F_2, \dots, F_n$ are constants.

In the field of convex optimization, a **linear matrix inequality (LMI)** is defined as an expression of the form:

$$A(y) = A_0 + y_1 A_1 + y_2 A_2 + \cdots + y_n A_n \succeq 0,$$

where $y = (y_1, \dots, y_n)$ is a real vector, $A_0, A_1, A_2, \dots, A_n$ are symmetric matrices, and $A(y) \succeq 0$ is a generalized inequality, meaning that $A(y)$ is a positive semidefinite matrix.

This linear matrix inequality defines a convex constraint on the variable y . There are efficient numerical methods for determining the feasibility of an LMI (e.g., whether there exists a vector y such that $A(y) \succeq 0$), as well as for solving convex optimization problems with LMI constraints.

The Cholesky and $LDL^\top$ factorizations that underlie this oracle, together with the row-based witness construction and its $O(p^3)$ complexity, are detailed in the Appendix.

3.3.1. Example: Matrix Norm Minimization. Let $A(x) = A_0 + x_1 A_1 + \cdots + x_n A_n$. Problem $\min_x \|A(x)\|$ can be reformulated as

$$\begin{aligned} & \text{minimize} && \gamma, \\ & \text{subject to} && \begin{pmatrix} \gamma I_m & A(x) \\ A^\top(x) & \gamma I_n \end{pmatrix} \succeq 0. \end{aligned}$$

A binary search on γ can be used for this problem.

3.3.2. Example: Minimum Eigenvalue and Eigenvalue Optimization. A second canonical use of the LMI oracle is the eigenvalue optimization problem (EVP). Given a symmetric matrix-valued affine function $A(x) = A_0 + x_1 A_1 + \cdots + x_n A_n$, consider

$$\begin{aligned} & \text{minimize} && \gamma, \\ & \text{subject to} && A(x) \preceq \gamma I, \end{aligned}$$

which minimizes the largest eigenvalue $\lambda_{\max}(A(x))$ when $A(x)$ is symmetric. Reading the constraint as a linear matrix inequality,

$$F(x, \gamma) = \gamma I - A(x) \succeq 0,$$

the problem is convex in the augmented variable (x, γ) , because F is affine and the cone of positive semidefinite matrices is convex. The ellipsoid method searches over (x, γ) , and the oracle must decide whether $F(x_0, \gamma_0) \succeq 0$ at the queried point.

The oracle is exactly the row-based Cholesky factorization described in the Appendix. If the factorization $F_{:,p} = R_{:,p}^\top R_{:,p}$ encounters a non-positive pivot at row p , then $F(x_0, \gamma_0)$ is not positive semidefinite and the vector $v = R_{:,p}^{-1} e_p$ satisfies

$$v^\top F_{:,p}(x_0, \gamma_0) v < 0.$$

Thus v is a direction of negative curvature: it certifies that the quadratic form defined by F is not nonnegative and, equivalently, that $A(x_0)$ has an eigenvalue exceeding γ_0 . The separating cut is formed from the same witness,

$$(-v^\top \partial F_{:p,:p}(x_0, \gamma_0)v, -v^\top F_{:p,:p}(x_0, \gamma_0)v),$$

where the partial derivatives are $-\partial A_{:p,:p}/\partial x_i$ with respect to x_i and I with respect to γ . A binary search on γ can be layered on top of the feasibility oracle, in the same manner as the matrix norm minimization example, since the feasible set is monotone in γ .

Two specializations deserve mention. When $A(x)$ is diagonal with entries x_i , the constraint $A(x) \preceq \gamma I$ reduces to the linear inequalities $x_i \leq \gamma$, and the EVP becomes an ordinary convex program; this is the simplest instance in which the LMI oracle degenerates to a coordinatewise comparison. When γ is fixed and F is a single symmetric matrix, the oracle computes its smallest eigenvalue and returns an associated eigenvector as the witness, which is the scalar form of the negative-curvature certificate.

3.3.3. Random Field [25]. *Random field*, also known as *stochastic process*, can be regarded as an indexed family of random variables denoted as $\{Z(\mathbf{s}) : \mathbf{s} \in D\}$, where D is a subset of d -dimensional Euclidean space $\mathbb{R}^d$. To specify a stochastic process, the joint probability distribution function of any finite subset $(Z(\mathbf{s}_1), \dots, Z(\mathbf{s}_n))$ must be given in a consistent way, which is called *distribution* of the process. For ease of analysis, a random field is often assumed to be with *Gaussian* distribution and is called Gaussian random field.

A random field has several key properties useful in practical problems. The field is *stationary* under translations, or *homogeneous*, if the distribution is unchanged when the point set is translated. The field is *isotropic* if the distribution is invariant under any rotation of the whole points in the parameter space. We study the homogeneous isotropic field in this paper.

The *covariance* C and *correlation* R of a stochastic process are defined by:

$$C(\mathbf{s}_i, \mathbf{s}_j) = \text{cov}(Z(\mathbf{s}_i), Z(\mathbf{s}_j)) = \mathbb{E}[(Z(\mathbf{s}_i) - \mathbb{E}[Z(\mathbf{s}_i)])(Z(\mathbf{s}_j) - \mathbb{E}[Z(\mathbf{s}_j)])]$$

and

$$R(\mathbf{s}_i, \mathbf{s}_j) = C(\mathbf{s}_i, \mathbf{s}_j) / \sqrt{C(\mathbf{s}_i, \mathbf{s}_i)C(\mathbf{s}_j, \mathbf{s}_j)}$$

respectively for all $\mathbf{s}_i, \mathbf{s}_j \in D$, where $\mathbb{E}[Z(\mathbf{s})]$ denotes the expectation of $Z(\mathbf{s})$. Thus a process is homogeneous if C and R depend only on the separation vector $\mathbf{h} = \mathbf{s}_i - \mathbf{s}_j$. Furthermore, it is isotropic if C and R depend upon $\mathbf{h}$ only through its length h , i.e.,

$$C(\mathbf{s}_i, \mathbf{s}_j) = C(\mathbf{h}) = C(h),$$

$$(3.2) \quad R(\mathbf{s}_i, \mathbf{s}_j) = R(\mathbf{h}) = R(h) = C(h)/C(0).$$

If we denote $C(0)$, the variance of $Z(\mathbf{s})$, as σ^2 , then the relationship between covariance and correlation is $C(h) = \sigma^2 R(h)$.

When the two components are considered, the measurement data can still be regarded as a Gaussian random field, but the correlation function will have a discontinuity at the origin. We call this phenomenon “nugget effect” [11].

$$\begin{aligned} \min_{\kappa, p} \quad & \|\Omega(p) + \kappa I - Y\| \\ \text{subject to} \quad & \Omega(p) \succcurlyeq 0, \kappa \geq 0. \end{aligned}$$

Let $\rho(h) = \sum_{i=1}^n p_i \Psi_i(h)$, where the p_i are unknown coefficients to be fitted and the Ψ_i form a family of basis functions. The covariance matrix $\Omega(p)$ can be recast as:

$$\Omega(p) = p_1 F_1 + \cdots + p_n F_n,$$

where $\{F_k\}_{i,j} = \Psi_k(\|s_j - s_i\|_2)$.

3.3.4. Difference of Convex Structure and the 2Y Bound. The correlation-function estimation problem seeks coefficients p such that the parametric matrix

$$\Omega(p) = p_1 F_1 + \cdots + p_n F_n, \quad (F_k)_{ij} = \Psi_k(\|s_j - s_i\|_2),$$

fits a sample covariance Y in a likelihood sense while remaining positive semidefinite. With a nugget term κI , the Gaussian criterion is

$$f(\Omega) = \log \det(\Omega + \kappa I) + \text{Tr}((\Omega + \kappa I)^{-1} Y), \quad \Omega \succeq 0, \kappa \geq 0.$$

The first term is concave, being the logarithm of the determinant composed with an affine map, and the second term is convex, being the trace of the inverse of a positive definite matrix composed with $Y \succeq 0$. Consequently f is a difference of convex functions,

$$f(\Omega) = \underbrace{\text{Tr}(\Omega^{-1} Y)}_{\text{convex } h} - \underbrace{(-\log \det \Omega)}_{\text{convex } g},$$

and a difference of convex functions is in general neither convex nor concave. The ellipsoid method requires convexity, so it is necessary to determine the region on which f is convex.

Take a symmetric perturbation U and put $A = \Omega^{-1/2} U \Omega^{-1/2}$ and $B = \Omega^{-1/2} Y \Omega^{-1/2}$. The second directional derivative of f along U is

$$D^2 f(U) = \text{Tr}(A^2(2B - I)),$$

which is nonnegative for every U if and only if $2B \succeq I$, that is,

$$\Omega \preceq 2Y.$$

Hence f is convex exactly on the region $\Delta_{2Y} = \{\Omega : 0 \prec \Omega \preceq 2Y\}$, and it possesses genuine negative curvature outside it. The boundary $\Omega = 2Y$ is not an arbitrary device: it is the edge of the convexity region of the Gaussian log-likelihood. Along the commuting ray $\Omega = tY$ the objective collapses to a single variable,

$$f(t) = n \log t + \log \det Y + \frac{n}{t}, \quad f''(t) = \frac{n(2-t)}{t^3},$$

which is convex for $t < 2$ and concave for $t > 2$, confirming that the sign flip occurs precisely at $\Omega = 2Y$. The same characterization appears in the statistical literature on maximum likelihood for linear Gaussian covariance models, where the likelihood is strictly concave in and only in Δ_{2Y} .

The region Δ_{2Y} is a trust region for the optimization, valid when the maximum likelihood estimate lies inside it, which holds with high probability when the model is correctly specified and Y is positive definite. It is not a constraint of the estimation problem, and the distinction is decisive: imposing $0 \preceq \Omega \preceq 2Y$ as a hard feasibility cut replaces the true estimate by its projection onto the trust region whenever the family estimate falls outside. Under misspecification the constrained solution is therefore biased toward the boundary, and no amount of additional data removes the bias if the model family cannot reproduce the true correlation.

4. Ellipsoid Method Revisited. The ellipsoid method was introduced independently by Yudin and Nemirovskii [32] and by Shor [27]. Khachiyan [13] used it to show that linear programming is solvable in polynomial time, settling the long-standing question of its theoretical complexity; see [3] for a survey. In practice, however, the simplex method is much faster, despite its exponential worst-case complexity.

4.1. Basic Ellipsoid Method. An ellipsoid $\mathcal{E}_k(x_k, P_k)$ is specified as a set

$$\{x \mid (x - x_k)P_k^{-1}(x - x_k) \leq 1\},$$

where $x_k \in \mathbb{R}^n$ is the center of the ellipsoid and $P_k \in \mathbb{R}^{n \times n}$ is a positive definite matrix.

Example: For a 2D ellipsoid centered at (0,0) with $P = \begin{bmatrix} 4 & 0 \\ 0 & 1 \end{bmatrix}$, the set would be all points (x, y) satisfying $x^2/4 + y^2 \leq 1$.

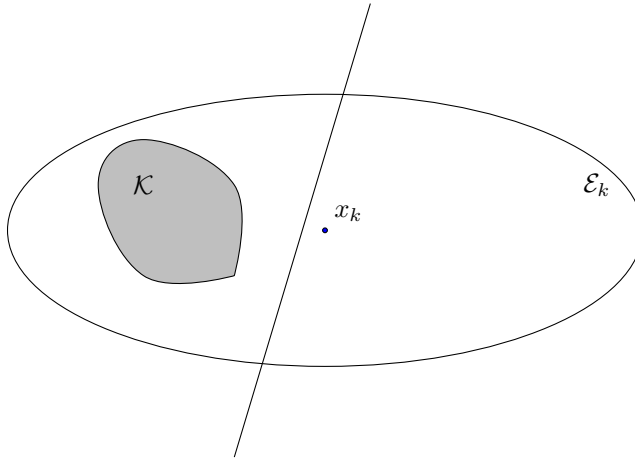


Figure 4.1: Ellipsoid, feasible region, and cut

Updating the ellipsoid (deep-cut)

Calculation of minimum volume ellipsoid covering:

$$\mathcal{E}_k \cap \{z \mid g^\top(z - x_k) + \beta \leq 0\}$$

Let $\tilde{g} = P_k g$, $\tau^2 = g^\top P_k g$. We can make the following observations:

1. If $n \cdot \beta < -\tau$ (shallow cut), then no smaller ellipsoid can be found.
2. If $\beta > \tau$, then intersection is empty.
3. Otherwise,

$$x_c^+ = x_c - \frac{\rho}{\tau^2} \tilde{g}, \quad P^+ = \delta \cdot \left(P - \frac{\sigma}{\tau^2} \tilde{g} \tilde{g}^\top \right)$$

where

$$\rho = \frac{\tau + nh}{n + 1}, \quad \sigma = \frac{2\rho}{\tau + \beta}, \quad \delta = \frac{n^2(\tau^2 - \beta^2)}{(n^2 - 1)\tau^2}$$

Example: For $n = 2$, $\tau = 2$, $\beta = 1$: $\rho = (2 + 2 * 1)/3 = 1.33$, $\sigma = 2 * 1.33/3 = 0.89$, $\delta = 4 * (4 - 1)/(3 * 4) = 1$

Even better, split P into two factors, κ and Q . Let $\tilde{g} = Qg$, $\omega = g^\top \tilde{g}$, and $\tau = \sqrt{\kappa\omega}$.

$$x_c^+ = x_c - \frac{\rho}{\omega} \tilde{g}, \quad Q^+ = Q - \frac{\sigma}{\omega} \tilde{g} \tilde{g}^\top, \quad \kappa^+ = \delta \cdot \kappa$$

Reduce n^2 multiplications per iteration. Note that:

- The determinant of Q decreases monotonically.
- The range of δ is $(0, n^2/(n^2 - 1))$.

4.1.1. The Ellipsoid as a Search Space and Its Initialization. The cutting-plane framework leaves the choice of search space open, and the efficiency of the method depends on three requirements: the set must be representable with few parameters, it must admit a closed-form update, and it must be guaranteed to contain the feasible region after every cut. The ellipsoid meets all three. It is the minimum-volume body, among those described by a quadratic form, that is (i) specified by $O(n^2)$ parameters, (ii) updated by explicit formulas, and (iii) known to contain the admissible set by induction. The first property distinguishes it from a general polyhedron, whose constraint count can grow without bound as cuts accumulate; the second distinguishes it from an arbitrary convex body, for which the minimum-volume enclosing update has no closed form.

In the split representation used throughout this article, the current ellipsoid is

$$\mathcal{E} = \{ x \mid (x - x_c)^\top Q^{-1} (x - x_c) \leq \kappa \},$$

where x_c is the center, Q is symmetric positive definite and encodes the shape, and $\kappa > 0$ is the scale factor. The two pieces play distinct roles: the eigenvectors of Q give the principal axes and the eigenvalues give the squared axis lengths up to the factor κ . Equivalently, the i -th semi-axis has length $\sqrt{\kappa \lambda_i(Q)}$, where $\lambda_i(Q)$ is the i -th eigenvalue, and the volume is

$$\text{vol}(\mathcal{E}) = \kappa^{n/2} \sqrt{\det Q} \text{vol}(\mathcal{B}^n),$$

with $\mathcal{B}^n$ the unit ball. The product $\kappa^{n/2}\sqrt{\det Q}$ is the quantity tracked by the convergence analysis, and it explains why the update is expressed as a multiplicative change of κ together with a rank-one modification of Q .

The initial ellipsoid must be certified to contain the feasible set, and two constructions are standard. If a bound R on the solution norm is known, the ball $Q = I$, $\kappa = R^2$ centered at the origin suffices. If a bounding box $l \leq x \leq u$ is known, the center is taken at the midpoint and a ball of radius $\|u - l\|_2/2$, that is $Q = I$ and $\kappa = (\|u - l\|_2/2)^2$, contains the entire box and therefore the feasible set. Because each iteration multiplies the volume by a factor bounded away from one, enlarging the initial volume by a factor V adds only $2n \ln V$ iterations, so a conservative initialization costs logarithmically; an initialization that fails to contain the feasible set, by contrast, invalidates the method outright. This asymmetry is the reason a safe but loose starting ellipsoid is usually preferred to an aggressive one.

4.1.2. Numerical Drift and the Stable $LDL^\top$ Variant. The representation above updates Q directly and relies on the accumulation of rank-one downdates to preserve positive definiteness. In exact arithmetic it does; in floating-point arithmetic it need not. Over the thousands of iterations that a typical problem requires, rounding errors in the repeated rank-one updates can drive an eigenvalue of Q through zero, after which Q is no longer positive definite, the scalar $\omega = g^\top Q g$ can become negative or denormal, and the iteration produces meaningless points or non-finite values. The failure is silent: the method keeps iterating and eventually returns a point that does not satisfy the constraints.

A numerically stable variant avoids this by maintaining a factorization rather than the full matrix. The shape is written as

$$Q = \kappa L D L^\top,$$

where L is unit lower triangular and D is diagonal. A cut is applied by three triangular sweeps,

$$w = L^{-1}g, \quad z = D^{-1}w, \quad \omega = w^\top z, \quad q = L^{-\top}z, \quad x_c \leftarrow x_c - \frac{\rho}{\omega} q,$$

followed by a rank-one update of the pair (L, D) that modifies those factors directly, in the manner of Gill, Murray, and Wright. The cut parameters (ρ, σ, δ) are computed from the same scalar $\tau^2 = \kappa \omega$ as before, so the two representations realize the same sequence of cuts in exact arithmetic. The advantage is structural: because the update preserves the unit lower-triangular form of L and the positivity of D , positive definiteness of Q is enforced by construction instead of being left to chance.

The stable variant is implemented with pre-allocated scratch buffers, so that the three sweeps and the rank-one update allocate nothing inside the iteration. This matters because the stable path performs more scalar operations than the direct one, and the per-iteration allocations that a naive implementation incurs can dominate its cost.

Empirically the two representations agree closely. On a suite of continuous, robust, and parallel-cut problems the iterates coincided to within 10^{-6} in the infinity norm, and the iteration counts differed by at most a fraction of a percent, for instance 83 versus 83 on a two-dimensional profit problem and 26,027 versus 26,125 on a thirty-two-tap parallel-cut filter. The per-iteration cost is likewise comparable in compiled languages: the stable-to-direct ratio ranged from about 1.07 to 1.4 across dimensions up to 256,

and at the largest dimension the stable variant was sometimes the faster of the two. The interpreted setting is different, because there the triangular sweeps are written as scalar loops; unless those loops are vectorized, the stable variant can be more than a hundred times slower at dimension 64, and the ratio grows with the dimension. The guidance is therefore to prefer the stable variant in compiled code, where it is near-free insurance against numerical drift; to keep the direct representation for rapid prototyping and for interpreted settings in which it has not been vectorized; and to regard the factorization choice as independent of the cut formulas, since the cuts themselves are identical.

Table 4.1: Comparison of the direct and the factorized ellipsoid representations. The cut parameters, and hence the iterates, are the same; only the arithmetic differs.

Representation	State	Update	Positive definiteness
Direct	Q	matrix-vector product then rank-one downdate	relied upon, not enforced
Stable	(L, D) , with $Q = \kappa LDL^\top$	triangular sweeps then rank-one update of (L, D)	preserved by construction

4.2. Central Cut. This is the special case $\beta = 0$. It deserves a separate implementation because it is much simpler. Let $\tilde{g} = Qg$, $\tau = \sqrt{\kappa \cdot \omega}$,

$$\rho = \frac{\tau}{n+1}, \quad \sigma = \frac{2}{n+1}, \quad \delta = \frac{n^2}{n^2-1}.$$

Example: For $n = 3$, $\tau = 2$: $\rho = 2/4 = 0.5$, $\sigma = 2/4 = 0.5$, $\delta = 9/8 = 1.125$

4.2.1. Complete Update Parameters for Central, Deep, and Parallel Cuts. All three cut types share one update map,

$$x_c^+ = x_c - \frac{\rho}{\omega} \tilde{g}, \quad Q^+ = Q - \frac{\sigma}{\omega} \tilde{g} \tilde{g}^\top, \quad \kappa^+ = \delta \kappa,$$

and differ only in the scalar triple (ρ, σ, δ) . Throughout,

$$\tilde{g} = Qg, \quad \omega = g^\top \tilde{g} = g^\top Qg > 0, \quad \tau = \sqrt{\kappa \omega}.$$

The parameter written h in the deep-cut formula quoted above is the cut offset β ; the two symbols should be identified, so that $\rho = (\tau + n\beta)/(n+1)$.

A single reference for the symbols shared by every cut type is Table 4.2.

Table 4.2: Unified notation for the ellipsoid update, shared by the central, deep, and parallel cuts.

Symbol	Quantity	Role
g	cut normal	subgradient of the violated constraint
β	cut offset	depth of the cut; the symbol h quoted above denotes the same quantity
$\tilde{g}$	Qg	the normal mapped through the shape
ω	$g^\top \tilde{g}$	positive scalar measuring g in the shape metric
τ	$\sqrt{\kappa \omega}$	ellipsoid radius along g
ρ	center-displacement factor	$x_c^+ = x_c - (\rho/\omega)\tilde{g}$
σ	rank-one factor	$Q^+ = Q - (\sigma/\omega)\tilde{g}\tilde{g}^\top$
δ	scale factor	$\kappa^+ = \delta\kappa$
Q, κ	shape and scale	the split representation of §4.1.1

Central cut ($\beta = 0$):

$$\rho = \frac{\tau}{n+1}, \quad \sigma = \frac{2}{n+1}, \quad \delta = \frac{n^2}{n^2-1}.$$

Parallel central cut (one plane through the center and one offset by β_1): with $\alpha^2 = \beta_1^2/\tau^2$, $k = (n/2)\alpha^2$, and $r = k + \sqrt{k^2 + 1 - \alpha^2}$,

$$\rho = \frac{\beta_1}{r+1}, \quad \sigma = \frac{2}{r+1}, \quad \delta = \frac{r}{r-1/n}.$$

Deep (bias) cut ($\beta > 0$): with $\eta = \tau + n\beta$,

$$\rho = \frac{\eta}{n+1}, \quad \sigma = \frac{2\eta}{(n+1)(\tau+\beta)}, \quad \delta = \frac{n^2}{n^2-1} \cdot \frac{\tau^2 - \beta^2}{\tau^2}.$$

The intersection is empty when $\beta > \tau$; the cut is too shallow for a strictly smaller enclosing ellipsoid to exist when $n\beta < -\tau$; and the expression above applies for $-\tau/n \leq \beta \leq \tau$. In the optimization setting a bias cut arises with $\beta = f_0(x_0) - \gamma$ whenever the queried point fails the current sublevel set.

Parallel-bias cut (two planes at β_0 and β_1): the complete formulation, and the one used by the reference implementations, is written in terms of the auxiliary quantities

$$\zeta_0 = \tau^2 - \beta_0^2, \quad \zeta_1 = \tau^2 - \beta_1^2, \quad \xi = \sqrt{\zeta_0 \zeta_1 + \left(\frac{n}{2}(\beta_1^2 - \beta_0^2)\right)^2}.$$

With $\eta = \tau^2 + n\beta_0\beta_1$,

$$\sigma = \frac{2\eta}{\tau^2 + \beta_0\beta_1 + \frac{n}{2}(\beta_0 + \beta_1)^2 + \xi}, \quad \rho = \sigma \cdot \frac{\beta_0 + \beta_1}{2},$$

Algorithm 3 Ellipsoid update (central, deep, or parallel cut)

Require: shape $Q \succ 0$, scale κ , center x_c , cut (g, β)

Ensure: updated (Q, κ, x_c)

- 1: $\tilde{g} \leftarrow Qg; \quad \omega \leftarrow g^\top \tilde{g}; \quad \tau \leftarrow \sqrt{\kappa \omega}$
 - 2: $(\rho, \sigma, \delta) \leftarrow \text{CUTPARAMS}(\beta, \tau, n)$
 - 3: $x_c \leftarrow x_c - (\rho/\omega) \tilde{g}$
 - 4: $Q \leftarrow Q - (\sigma/\omega) \tilde{g} \tilde{g}^\top$
 - 5: $\kappa \leftarrow \delta \kappa$
 - 6: **return** (Q, κ, x_c)
-

$$\delta = \frac{n^2}{(n^2 - 1) \tau^2} \left(\frac{\zeta_0 + \zeta_1}{2} + \frac{\xi}{n} \right).$$

This form improves on the expression quoted earlier in terms of the mean offset and its reciprocal square. It remains finite when the two offsets are symmetric, $\beta_0 + \beta_1 = 0$, a configuration for which that earlier expression is undefined and which the accompanying example explicitly flags. It also degenerates correctly: setting $\beta_1 = \tau$ makes the second plane tangent to the ellipsoid, and the parameters then reduce exactly to the deep-cut values with $\beta = \beta_0$. The familiar cases carry over: the intersection is empty when $\beta_0 > \beta_1$; no smaller enclosing ellipsoid exists when $\beta_0 \beta_1 < -\tau^2/n$; and the update reduces to the deep cut when $\beta_1^2 > \tau^2$.

Volume reduction. Writing $Q^+ = Q - (\sigma/\omega) \tilde{g} \tilde{g}^\top$ and applying the matrix-determinant lemma,

$$\det Q^+ = \det Q \left(1 - \frac{\sigma}{\omega} g^\top Q g \right) = (1 - \sigma) \det Q,$$

so $\det Q$ decreases monotonically, because $\sigma \in (0, 1)$ for every admissible cut. The volume ratio is therefore

$$\frac{\text{vol}(\mathcal{E}^+)}{\text{vol}(\mathcal{E})} = \delta^{n/2} (1 - \sigma)^{1/2} \leq e^{-1/(2n)},$$

the inequality holding for central, deep, and parallel cuts alike. After k iterations the volume is at most $e^{-k/(2n)}$ times the initial volume, so reducing it to a fraction ϵ requires $k \approx 2n \ln(1/\epsilon)$ iterations; for $n = 10$ and $\epsilon = 10^{-6}$ this is roughly 276 iterations. The factor δ lies in $(0, n^2/(n^2 - 1))$, approaching its upper end as the cut becomes central.

4.3. Parallel Cuts. The oracle returns a pair of cuts instead of a single cut. The pair is given by g and (β_1, β_2) such that:

$$\begin{aligned} g^\top(x - x_c) + \beta_1 &\leq 0, \\ g^\top(x - x_c) + \beta_2 &\geq 0, \end{aligned}$$

for all $x \in \mathcal{K}$.

Only a linear inequality can produce such a parallel cut:

$$l \leq a^\top x + b \leq u, \quad L \preceq F(x) \preceq U.$$

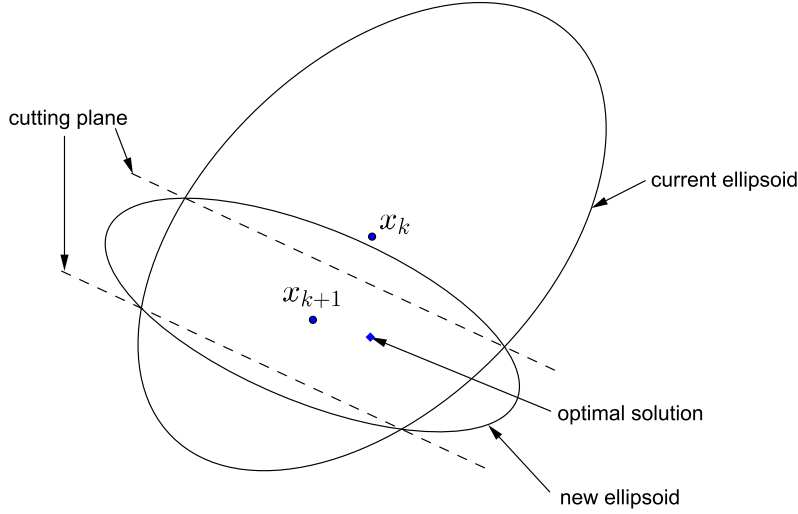


Figure 4.2: Parallel cuts

They usually provide faster convergence.

Updating the ellipsoid.

Let $\tilde{g} = Qg$, $\tau^2 = \kappa \cdot \omega$.

- If $\beta_1 > \beta_2$, intersection is empty.
- If $\beta_1\beta_2 < -\tau^2/n$, no smaller ellipsoid can be found.
- If $\beta_2^2 > \tau^2$, it reduces to a deep cut with $\beta = \beta_1$.

Otherwise,

$$x_c^+ = x_c - \frac{\rho}{\omega} \tilde{g}, \quad Q^+ = Q - \frac{\sigma}{\omega} \tilde{g} \tilde{g}^\top, \quad \kappa^+ = \delta \kappa.$$

where

$$\begin{aligned} \bar{\beta} &= (\beta_1 + \beta_2)/2, \\ \xi^2 &= (\tau^2 - \beta_1^2)(\tau^2 - \beta_2^2) + (n(\beta_2 - \beta_1)\bar{\beta})^2, \\ \rho &= \bar{\beta} \sigma, \\ \delta &= \frac{n^2}{(n^2-1)\tau^2} \left(\tau^2 - \frac{\beta_1^2 + \beta_2^2}{2} + \frac{\xi}{n} \right), \end{aligned}$$

with σ taken from the ζ -formulation of §4.2.1. That formulation involves no mean offset and therefore remains finite when $\bar{\beta} = 0$; a mean-offset expression would divide by $\bar{\beta}^2$ and fail in that symmetric case.

Example: for $n = 2$, $\tau = 2$, $\beta_1 = -1$, and $\beta_2 = 1$, the mean offset vanishes, $\bar{\beta} = 0$, so a mean-offset formula for σ is undefined. The ζ -formulation of §4.2.1 instead yields a finite value, which is exactly why that symmetric case needs special handling.

4.3.1. Example: FIR filter design. A typical structure of a digital finite impulse response (FIR) filter is shown in Fig. 4.3, where the coefficients $h[0], h[1], \dots, h[n-1]$ must be determined to meet given specifications. Usually, they can be manually designed using windowing or frequency-sampling techniques [21].

However, such methods rely heavily on the designer's experience and offer no guarantee of quality. Optimization-based techniques (e.g., [30]) have therefore attracted considerable research effort, and with growing computing power the solution space can be explored effectively.

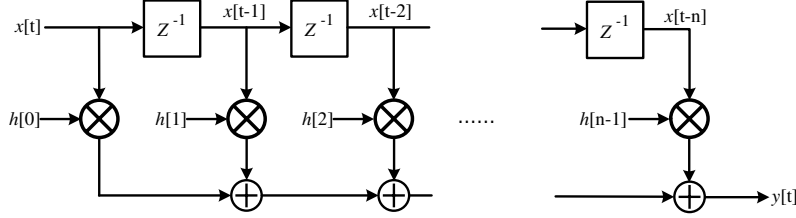


Figure 4.3: A typical structure of an FIR filter [20].

In optimization algorithms, what is particularly interesting is the convex optimization. If a problem is in a convex form, it can be efficiently and optimally solved. Convex optimization techniques are also implementable in designing FIR filters, including the Parks-McClellan algorithm [23], METEOR [28], and peak-constrained least-squares (PCLS) [1, 26]. In the mentioned articles, with the help of exchange algorithms (e.g. Remez exchange algorithm), certain FIR filter design problems can be formed as linear or quadratic programs. These are two simple forms of convex optimization, solvable by existing algorithms such as the interior-point method [4]. Motivated by the resulting optimality, much effort has gone into casting the design problem as a convex one; in particular, [30] uses spectral decomposition [12] to formulate FIR design with frequency-domain magnitude constraints as a convex program. More examples are provided in [10].

Its time response is

$$(4.1) \quad y[t] = \sum_{k=0}^{n-1} h[k]u[t-k]$$

where $\mathbf{h} = (h(0), h(1), \dots, h(n-1))$ are the filter coefficients. Its frequency response $H : [0, \pi] \rightarrow \mathbb{C}$ is

$$(4.2) \quad H(\omega) = \sum_{m=0}^{n-1} h(m)e^{-jm\omega}$$

where $j = \sqrt{-1}$, n is the order of the filter. The design of a filter with magnitude constraints is often formulated as a constrained optimization problem of the form

$$\begin{aligned} & \min \gamma \\ & \text{subject to } f(\mathbf{x}) \leq \gamma \\ & \quad g(\mathbf{x}) \leq 0. \end{aligned}$$

{\#eq:ori} where $\mathbf{x}$ is the vector of design variables, $g(\mathbf{x})$ represents the desired filter characteristics and $f(\mathbf{x})$ is the performance metric to be optimized. For example, the magnitude constraints on frequency domain are expressed as

$$(4.3) \quad L(\omega) \leq |H(\omega)| \leq U(\omega), \forall \omega \in (-\infty, +\infty)$$

where $L(\omega)$ and $U(\omega)$ are the lower and upper (nonnegative) bounds at frequency ω respectively. Note that $H(\omega)$ is 2π periodic and $H(\omega) = \overline{H(-\omega)}$. Therefore, we can only consider the magnitude constraint on $[0, \pi]$ [30].

Generally, the problem might be difficult to solve, since we can only obtain the global optimal solution with resource-consuming methods, such as branch-and-bound [10]. However, the situation is totally different if the problem is convex, where $f(\mathbf{x})$ and $g(\mathbf{x})$ are convex functions. In such a case, the problem can be optimally solved with many efficient algorithms.

Attracted by the benefits, the authors of [30] transformed the originally non-convex problem into a convex form via spectral decomposition:

$$(4.4) \quad L^2(\omega) \leq R(\omega) \leq U^2(\omega), \forall \omega \in (0, \pi)$$

where $R(\omega) = \sum_{t=-n+1}^{n-1} r(t)e^{-j\omega t} = |H(\omega)|^2$ and $\mathbf{r} = (r(-n+1), r(-n+2), \dots, r(n-1))$ are the autocorrelation coefficients. Especially, $\mathbf{r}$ can be determined by $\mathbf{h}$, and vice versa [30]:

$$(4.5) \quad r(t) = \sum_{i=-n+1}^{n-1} h(i)h(i+t), t \in \mathbb{Z}.$$

where $h(t) = 0$ for $t < 0$ or $t > n-1$.

4.3.2. Example: Maximum Likelihood estimation. Consider

$$\begin{aligned} \min_{\kappa, p} \quad & \log \det(\Omega(p) + \kappa \cdot I) + \text{Tr}((\Omega(p) + \kappa \cdot I)^{-1}Y), \\ \text{subject to} \quad & \Omega(p) \succeq 0, \kappa \geq 0. \end{aligned}$$

Note that the first term is concave, the second term is convex. However, if there are enough samples such that Y is a positive definite matrix, then the function is convex within $[0, 2Y]$. Therefore, the following problem is convex:

$$\begin{aligned} \min_{\kappa, p} \quad & \log \det V(p) + \text{Tr}(V(p)^{-1}Y), \\ \text{subject to} \quad & \Omega(p) + \kappa \cdot I = V(p) \\ & 0 \preceq V(p) \preceq 2Y, \kappa > 0. \end{aligned}$$

4.3.3. Convex-Concave Procedure and Numerical Caveats. Because $f = h - g$ is a difference of convex functions, it can be minimized by the convex-concave procedure (CCP), also known as majorize-minimize. At the current iterate Ω_k , the

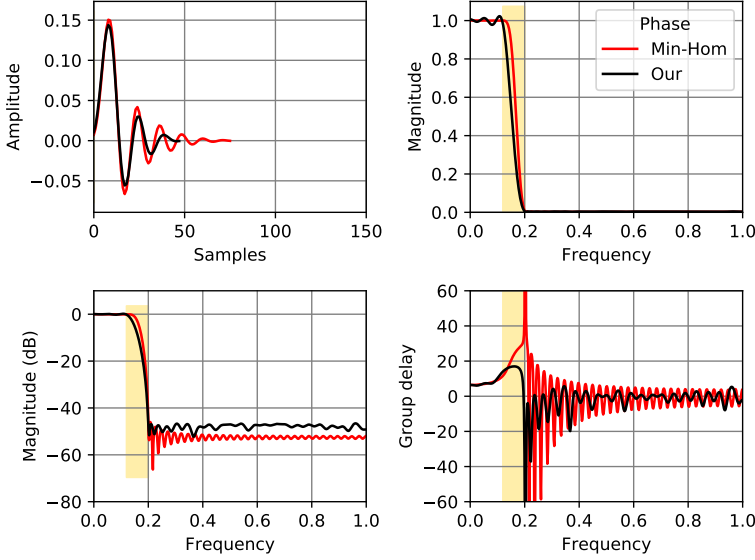


Figure 4.4: Result

concave term is majorized by its tangent, which for a concave function lies above the graph:

$$g(\Omega) \leq g(\Omega_k) + \text{Tr}(\Omega_k^{-1}(\Omega - \Omega_k)) = \log \det \Omega_k + \text{Tr}(\Omega_k^{-1}(\Omega - \Omega_k)).$$

Substituting this upper bound gives the convex surrogate

$$S_k(\Omega) = \text{Tr}(\Omega^{-1}Y) + \log \det \Omega_k + \text{Tr}(\Omega_k^{-1}(\Omega - \Omega_k)) \geq f(\Omega),$$

whose gradient at Ω_k equals $\nabla f(\Omega_k)$. Dropping the constant terms, the subproblem solved at each step is

$$\min_{\Omega \succ 0} \text{Tr}(\Omega^{-1}Y) + \text{Tr}(M_k\Omega), \quad M_k = \Omega_k^{-1},$$

a convex problem that the cutting-plane method can solve with the same LMI oracle. Because the surrogate is a majorant that touches f at the current iterate, the iterates are monotone,

$$f(\Omega_{k+1}) \leq S_k(\Omega_{k+1}) \leq S_k(\Omega_k) = f(\Omega_k),$$

and a fixed point of the procedure is a stationary point of the true likelihood. No trust region and no $2Y$ bound are imposed; instead, the non-convexity is handled by a sequence of convex problems. This is the appropriate remedy when the family estimate lies outside Δ_{2Y} , where the direct application of the ellipsoid method would otherwise be invalid.

A numerical caveat applies to the evaluation of the surrogate. The gradient of the likelihood requires $S = \Omega^{-1}$, and it is tempting to form $S = R^{-\top}R^{-1}$ from the upper triangular factor R produced by a Cholesky decomposition $\Omega = R^{\top}R$. The inverse of

a triangular matrix must be computed by triangular back-substitution; using a general symmetric positive definite inverse routine silently treats R as symmetric and, because the off-diagonal terms of a triangular matrix are discarded, returns $\text{diag}(1/R_{ii})$ instead of R^{-1} . Both the objective and its gradient are then corrupted, and the solver may converge to a point that is stationary for the wrong function. Triangular factors should always be inverted with a triangular routine, and the result should be checked against the identity $\Omega\Omega^{-1} = I$ in a regression test.

4.4. Implementation. The cutting-plane framework separates naturally into three components with distinct responsibilities: a *search space* that carries all mutable state, an *oracle* that encodes the problem, and a *generic algorithm* that only passes information between the two. This separation is not merely organizational. It is what allows a single, problem-agnostic iteration loop to serve feasibility, continuous optimization, quantized optimization, and one-dimensional binary search, while the problem-specific knowledge remains confined to the oracle.

The search space is the only stateful object. It retains the current center and the shape of the ellipsoid, and it exposes a small interface: report the center, report a scalar measure of the current size, replace the center, and apply a cut. Applying a cut is the only computationally significant operation, since it amounts to a rank-one update of the shape together with a displacement of the center. Different representations of the shape may be substituted without touching the algorithm: a dense ellipsoid, a factorized ellipsoid that maintains an LDL^T decomposition for numerical stability, or, in one dimension, a single interval described only by its center and radius. Because the algorithm depends on the interface rather than on the representation, the same loop drives all of them.

The oracle is the problem-specific interface. Conceptually, an oracle is queried at a point and answers in one of two ways: the point is acceptable, or the point is rejected and a cutting plane is supplied that separates it from the admissible set. Four variants of this idea suffice for the applications considered here. A *feasibility oracle* answers membership and returns a single cut when the query is rejected. An *optimization oracle* additionally receives a mutable best-so-far objective value; when the query improves that value it reports an improvement, which directs the algorithm to take a central cut, and otherwise it reports no improvement, which directs a deeper cut. A *quantized optimization oracle* applies the same logic to a nearby admissible discrete point rather than to the query itself, and additionally reports whether alternative cuts remain, so that a cut with no effect can be retried. A *binary-search oracle* answers only whether a candidate objective level is attainable; it is realized by wrapping a feasibility oracle and running a nested feasibility problem at each level, which makes the classical bisection over a quasiconvex objective a special case of the same interface.

The type of cut is carried by the oracle and determines the update rule. A *single cut* is described by one offset, while a *parallel cut* is described by a pair of offsets and is appropriate whenever the constraint is a two-sided inequality, as for the magnitude bounds of the filter design. Central and deep cuts are the special cases of a single cut distinguished by the sign of the offset. Making the cut type explicit, rather than passing bare scalars or tuples, has two benefits: the update strategy and the oracle are instantiated with a common cut type, so a mismatch cannot occur, and the mapping from a problem constraint to a cut geometry is documented at the level of the interface.

Each update returns an *update status*, and the algorithm branches on it. A status of

success means the ellipsoid was reduced and the iteration continues. A status of no solution means the intersection is empty, which certifies infeasibility and terminates the loop early. A status of no effect means the cut failed to reduce the ellipsoid, which is possible with a shallow quantized cut and triggers a retry when an alternative discrete point is available. A status of unknown covers numerical failures. The generic loops for feasibility, optimization, and quantized optimization differ only in how they combine the oracle reply with the update status; the arithmetic of the ellipsoid itself never appears in them. On a continuous problem, the quantized variant converged in roughly one third of the iterations of the continuous variant, since the admissible discrete set is smaller, and the separation of concerns permitted this reuse without duplicating the algorithm.

4.5. Performance Considerations. The cost of the method is dominated by the per-iteration work. Each iteration forms the matrix-vector product $\tilde{g} = Qg$, evaluates the scalar $\omega = g^\top \tilde{g}$, and performs a rank-one correction of the shape together with a displacement of the center; only the matrix-vector product is of order n^2 . The number of iterations is typically between 10^3 and 10^5 , so the constant factor multiplying this work matters as much as the asymptotic rate. Three practical lessons emerge from a systematic study of implementations in three languages.

First, the bottleneck is rarely where it is expected. Profiling an interpreted implementation showed that the oracle, and not the ellipsoid update, dominated the runtime. The oracle evaluated one short scalar product per constraint, and the sheer number of such calls, each carrying full interpreter dispatch, outweighed the arithmetic they performed. The remedy was to replace the per-constraint callbacks with a single matrix-vector product covering all candidate constraints of a band, followed by boolean masks that locate the first violation. The scan is still performed lazily, so later bands are skipped once an earlier band violates, and the round-robin cursor that determines which constraint is examined next is preserved exactly, since that cursor is what prevents the same cut from being reintroduced indefinitely. On the scan primitive itself, this change reduced the cost by more than an order of magnitude. The lesson is that in interpreted code the cost of the call is usually the cost, not the arithmetic inside it.

Second, per-iteration allocation is a recurring defect. In C++, expression templates over dynamic arrays create temporary vectors for each arithmetic expression, so a sequence of vector operations allocates several temporaries per iteration; writing the same computation as explicit loops into pre-allocated buffers removes them. A related defect is allocating the gradient buffer inside the outer wrapper on every call, even though the numerically stable ellipsoid variant already used a persistent scratch buffer. In the matrix-inequality oracle, a fresh cut vector was constructed on every rejected query; reusing a persistent cut buffer avoided the allocation and the copy. In Rust, a helper that returned a newly allocated vector on every iteration was the single largest cost identified; pre-allocating the destination and writing into it in place removed the tax. The same reasoning applies to the symmetric quadratic form used by the matrix-inequality oracle: summing over the whole matrix does twice the necessary work, whereas summing the upper triangle and doubling the off-diagonal contribution halves it. Finally, constructing array views and invoking a dot product on very short slices makes per-call overhead dominate, and descending to flat slices traversed by plain loops is faster.

Third, several intuitive ports of a successful optimization do not generalize. Flattening

a nested array to contiguous storage was slower, a single vector expression that allocated per row was much slower, and assembling a dense matrix in place of a lazy per-element evaluation was slower for small blocks. The reason is that the original scan stops at the first violated constraint, so the partial work it performs is less than the full matrix-vector product that replaces it. In compiled code, early exit can therefore beat vectorization, and a vectorization win observed in an interpreted language must be re-measured before it is ported. This is an instance of the more general rule that one should port the measurement, not the conclusion.

Taken together, these observations give transferable engineering guidance. Measure before optimizing, and capture a baseline honestly by alternating between the two configurations under identical conditions. Distinguish the cost of the number of calls from the cost of the arithmetic inside them; the former usually dominates in interpreted code and the latter in compiled code. Hoist constants that do not change between iterations. Pre-allocate and reuse every buffer that the hot loop touches, and never allocate inside it. Treat consistency across sibling implementations as an optimization in its own right, since the numerically stable variant was already free of the defect that had crept into the classic path. Preserve exact iteration counts on fixed test problems: an unchanged count is strong evidence that an optimization preserved behavior, even when last-bit floating-point noise slightly alters the trajectory. The illustrative speedups reported here range from roughly 1.3 to 5 times, depending on language and workload, and are specific to the authors’ implementations and machines; they indicate the magnitude of the allocation and call-count effects rather than universal constants.

The section’s guidance can be condensed into a short checklist.

Table 4.3: Engineering rules of thumb for embeddable oracles, distilled from the implementation study of this section.

Rule of thumb	Why it matters
Measure, then optimize; capture the baseline by alternating configurations under identical conditions.	Prevents crediting noise, or a port artifact, as a real gain.
Distinguish the cost of the calls from the cost of the arithmetic inside them.	The former dominates in interpreted code and the latter in compiled code; optimizing the wrong one wastes effort.
Pre-allocate and reuse every buffer the hot loop touches; never allocate inside it.	Per-iteration allocation was the single largest recurring defect across the C++, Rust, and Python implementations.
Prefer the factorized (stable) representation in compiled code; keep the direct one for prototyping or unvectorized interpreted use.	The stable form is near-free insurance when compiled, yet can exceed 100× slower when not.
Port the measurement, not the conclusion.	A vectorization win in one language or workload frequently fails to generalize.
Preserve exact iteration counts on fixed test problems.	An unchanged count is strong evidence that an optimization preserved behavior.

Rule of thumb	Why it matters
Pair an absolute stopping threshold with a stall test.	A threshold on a scale-dependent quantity cannot terminate at floating-point resolution and otherwise burns inner solves.

4.6. Comparison with Interior-Point Solvers. We now place the cutting-plane method alongside a mature interior-point solver on three affine-constraint problems, all of which admit exact cuts and therefore isolate the algorithmic and language overheads from oracle complexity. In the measurements below the interior-point solver is reached through the CVXPY modeling layer, while the interpreted and compiled cutting-plane implementations are the ellalgo and ellalgo-cpp packages. The first problem is the Chebyshev center, the largest Euclidean ball contained in a polyhedron

$$\mathcal{P} = \{ x \in \mathbb{R}^n \mid a_i^\top x \leq b_i, \ i = 1, \dots, m \},$$

where the design variables $(x, r) \in \mathbb{R}^{n+1}$ jointly maximize the radius r subject to the affine constraints

$$f_i(x, r) = a_i^\top x + \|a_i\|_2 r - b_i \leq 0, \quad i = 1, \dots, m.$$

Every constraint has the constant gradient $\partial f_i = (a_i, \|a_i\|_2)$, so a violated constraint yields an exact cut and no oracle error enters the comparison; this is the classical Chebyshev-center example of Section 8.5.1 in [4]. The second problem is the minimum-eigenvalue problem for an affine matrix pencil subject to box constraints,

$$\begin{aligned} & \text{minimize} && \gamma, \\ & \text{subject to} && A(x) \preceq \gamma I, \quad -1 \leq x_i \leq 1, \ i = 1, \dots, n, \end{aligned}$$

with $A(x) = A_0 + x_1 A_1 + \dots + x_n A_n$; equivalently $F(x, \gamma) = \gamma I - A(x) \succeq 0$, so the $LDL^\top$ witness described in §3.3 supplies the cut. The third problem is the lowpass filter design of §4.3, where the two-sided magnitude constraints $L^2(\omega) \leq R(\omega) \leq U^2(\omega)$ on the passband are exploited as parallel cuts. The first two are prototypical of the convex formulations that classical interior-point methods [4] handle with near-linear scaling, while the third is representative of the filter-design applications that motivated the spectral formulation of [30].

Across the three problems, a consistent pattern was observed. On the Chebyshev center, all implementations located the same center, agreeing to five significant digits and satisfying the constraints, and the compiled cutting-plane implementation was faster than the interior-point solver at every tested dimension, from $n = 5$ to $n = 30$; the interpreted implementation was competitive only at the smallest dimensions. On the minimum-eigenvalue problem, the compiled implementation was faster for $n \leq 16$, with the gap narrowing as the number of iterations grew, and the interior-point solver caught up near $n = 20$. On the lowpass design, the compiled cutting-plane implementation was faster at every filter length from $n = 24$ to $n = 80$, while the interior-point runtime grew steeply, by roughly a factor of 38 over that range, and the cutting-plane runtime remained nearly flat. Iteration counts matched between the interpreted and compiled cutting-plane implementations to within about one percent on the lowpass problem and about six percent on the matrix-inequality problem, indicating faithful ports; the

runtime differences are therefore dominated by the per-iteration constant rather than by the algorithm.

Table 4.4: Chebyshev-center benchmark: wall-clock time against the number of variables n , for a polyhedron with 70 constraints. Speedup is the ratio of the other runtime to the compiled runtime, so a value above one favors the compiled implementation.

n	Interior-point (s)	Interpreted (s)	Compiled (s)	Speedup vs interior	Speedup vs interpreted
5	0.0580	0.0619	0.000548	106×	113×
10	0.0785	0.2305	0.002348	33×	98×
15	0.1365	0.5001	0.007284	19×	69×
20	0.1571	1.2116	0.015754	10×	77×
30	0.2543	2.3567	0.060679	4×	39×

Table 4.5: Minimum-eigenvalue benchmark: wall-clock time against the design dimension n of the affine matrix pencil, subject to the box bounds $-1 \leq x_i \leq 1$. The interior-point solver overtakes the compiled cutting-plane implementation at $n = 20$.

n	Interior-point (s)	Interpreted (s)	Compiled (s)	Speedup vs interior	Speedup vs interpreted
5	0.0316	0.1122	0.000587	54×	191×
8	0.0182	0.2664	0.001433	13×	186×
12	0.0201	0.5480	0.004078	5×	134×
16	0.0208	1.1686	0.008815	2×	133×
20	0.0258	0.5061	0.029870	0.86×	17×

Table 4.6: Lowpass FIR benchmark: wall-clock time against the filter length n , with $m = 15n$ frequency samples in the constraint grid. Stopband gain is the largest stopband magnitude in decibels, so more negative is better.

n	Interior-point (s)	Interpreted (s)	Compiled (s)	Speedup vs interior	Speedup vs interpreted	Stopband gain (dB)
24	0.092	2.567	0.049	2×	52×	-20.4
32	0.135	4.402	0.109	1×	40×	-33.8
48	0.446	3.433	0.117	4×	29×	-53.6
64	0.910	2.629	0.097	9×	27×	-73.6
80	3.473	4.277	0.191	18×	22×	-79.6

For the Chebyshev center the three implementations returned the same ball to five significant figures: the interior-point solver produced $r^* = 0.397723$ and both cutting-plane implementations $r^* = 0.397715$, a relative difference of 2×10^{-5} , with the largest constraint violation at the level of the floating-point tolerance, 10^{-6} or less. The

agreement is not merely numerical; it is a consequence of the affine formulation, in which every violated constraint gives a perfect cut. The iteration counts, however, differ by two orders of magnitude: the ellipsoid method required 1,854 iterations where the interior-point solver required about ten, and the compiled cutting-plane implementation was nevertheless faster, which locates the advantage in the per-iteration constant rather than in the iteration count. Over the three experiments the measured iteration count grows quadratically with the dimension, $N \approx 15 n^2$, so the total cost is set jointly by that count and the quadratic per-iteration work. The runtimes are indicative wall-clock measurements taken from the authors’ implementations on specific machines and are intended to display qualitative trends rather than portable constants.

The comparison exposes a genuine trade-off. The interior-point solver is declarative and self-scaling: the problem is described in a modeling language, the solver chooses its own iteration count, and it can exploit warm starts and near-linear scaling when the data are large. The cutting-plane method is algorithmic and requires the user to supply a separation oracle, an initial ellipsoid that contains the solution, and a dimension-aware iteration budget of order n^2 . In the experiments the initial ellipsoid is the ball of radius $R = \sqrt{n+1} + 1$ centered at the origin, that is $Q = I$ and $\kappa = R^2$ in the split representation of §4.1.1; the radius grows only as $\sqrt{n}$, so the initialization is inexpensive, but it must be chosen conservatively, because an initial ellipsoid that fails to contain the feasible set invalidates the method. In exchange the method never needs to evaluate all constraints, and it applies unchanged to constraint sets that are infinite or accessible only through an oracle.

One qualification is worth stating. The authoring of a separation oracle is no longer the barrier it once was: given a problem statement, a modern language model can draft the oracle together with its subgradient and its cut. What such assistance does not remove is the need to verify the oracle. The method is only as sound as the oracle it queries; an oracle that returns an invalid cut or a wrong subgradient yields a wrong answer, and the failure is invisible in the iteration count. The user’s effort therefore shifts from writing the oracle to testing it, by checking cut validity on known points, certifying feasibility independently, and comparing the subgradient against the analytic one.

This apparent reversal should be read with care, for several reasons. The comparison sets a research-grade implementation against a mature solver stack that has benefited from years of tuning, code generation, and sparse linear algebra, none of which the cutting-plane implementations have received. The interior-point path can be warm-started, whereas the ellipsoid method restarts from a containing ellipsoid. Tolerances and stopping criteria were not tuned to be identical. Most importantly, the language in which the method is expressed is a first-order effect: the per-iteration overhead of an interpreted environment inflates the cutting-plane method by roughly two orders of magnitude relative to compiled code, which is enough to reverse the ranking even when the compiled method is faster by more than an order of magnitude. A fair comparison should therefore be drawn within a single language, or else should isolate the per-iteration constant from the iteration count.

As a rule of thumb, the ellipsoid method is preferable when the constraints are available only through a separation oracle, when their number is large or infinite as in robust optimization, when the number of design variables is moderate, when some variables are discrete or quantized and the nearest admissible point is cheap to compute, when

a dependency-light or embeddable implementation is desired, or when a compiled language is available. Interior-point methods are preferable when the number of design variables is large, when all constraints can be evaluated explicitly and cheaply, when a mature modeling layer with warm-starting is available, when very high accuracy is required, or when the same problem must be re-solved many times as the data change. Neither method dominates, and the crossover depends on both the problem and the implementation language.

4.7. Numerical Experiments. The measurements below are indicative results from the authors’ implementations and display orders of magnitude rather than portable constants.

Table 4.7: Parallel-cut speedup on a lowpass FIR design: about $20\times$.

Variant	Time (ns)
single cut	627,743,505
parallel cut	30,497,546

The direct and the factorized (stable) ellipsoid agree to machine precision, and their iteration counts differ only marginally:

Table 4.8: Direct versus factorized (stable) ellipsoid on representative problems.

Case	Direct	Stable	$\ x_D - x_S\ _\infty$
Profit, $n = 2$	83	83	$< 10^{-6}$
Robust profit, $n = 2$	90	90	$< 10^{-6}$
Lowpass-32, parallel	26,027	26,125	$< 10^{-6}$
Lowpass-32, serial	40,740	40,621	$< 10^{-6}$
Lowpass-48	35,014	35,098	$< 10^{-6}$
Lowpass-64	27,805	27,926	$< 10^{-6}$

The relative cost of the stable form, the ratio of stable to direct per-iteration time, is close to one in compiled code and grows only in an interpreted implementation:

Table 4.9: Stable-to-direct per-iteration ratio on synthetic random cuts; a value below 1.0 means the stable form is faster.

Dimension n	Python	C++	Rust
16	11.1	1.07	1.55
32	37.6	1.19	1.86
64	122.3	1.38	2.01
128	–	1.07	0.90
256	–	1.12	0.77

4.8. Discrete Optimization. Many engineering problems, such as digital circuit sizing, can be formulated through convex or geometric programming. However, in ASIC design, there is frequently a limited number of cell types to select from in the cell library. This means that some design variables are discrete. Mapping the design variables to integers yields a mixed-integer convex programming (MICP) formulation.

What are the issues with existing methods? They are primarily based on relaxation. The more relaxed solution is used as the lower bound, and then the branch-and-bound method is applied to find the discrete optimal solution. The branch-and-bound method, however, does not exploit the convexity of the problem. What if only constraints regarding discrete data could be evaluated?

Typically, a relaxed (convex) optimum is computed first, and a discrete solution is then obtained by an exhaustive neighborhood search. However, tight constraints can cause a significant difference between the discrete and relaxed continuous optimal solutions. Enumerating the discrete domains can be challenging.

Consider:

$$\begin{aligned} & \text{minimize} && f_0(x), \\ & \text{subject to} && f_j(x) \leq 0, \forall j = 1, 2, \dots, \\ & && x \in \mathbb{D}, \end{aligned}$$

where $f_0(x)$ and $f_j(x)$ are “convex”. Note that some design variables are discrete. The oracle looks for a discrete solution x_d near x_c and cuts with:

$$g^\top(x - x_d) + \beta \leq 0, \beta \geq 0, g \neq 0.$$

Note that the cut may be a shallow cut. Use as many different cuts as possible in each iteration, for example by evaluating the constraints round-robin.

4.8.1. Discrete oracle internals. The construction of the discrete oracle expands the brief description above. The oracle is queried at a continuous center x_c , the current autocorrelation. It differs from the continuous oracle in four respects.

Mapping the center to the nearest discrete coefficient vector. The constraints do not act on the coefficients directly but through the autocorrelation, so the oracle first computes the continuous factor $\mathbf{h} = S(x_c)$ and then replaces every coefficient by its nearest CSD-representable value under the non-zero digit budget,

$$h_{\text{csd}}[k] = \text{csd}(h[k], \text{nnz}).$$

The proximity is measured in absolute value over the discrete set of canonical signed-digit numbers whose non-zero signed digit count is at most nnz . Because the coupling between coefficients is mediated by the autocorrelation, the coordinate-wise nearest vector need not be nearest in the induced response metric; the oracle therefore treats the result as a candidate and admits it only after exact evaluation.

The non-zero digit budget. The budget nnz bounds the number of non-zero signed digits per coefficient and is, up to common sub-expressions, the number of adders per tap. Values between three and five are typical. Enlarging the budget enlarges the discrete feasible set and therefore improves the attainable magnitude response at the

price of additional hardware; shrinking it to one or two digits renders many otherwise realizable filters infeasible at the required precision. The budget is a design constraint rather than a numerical parameter.

Exact re-evaluation of the candidate. The quantized autocorrelation is recomputed by the exact convolution $\mathbf{r}_{\text{csd}} = S^{-1}(\mathbf{h}_{\text{csd}})$, and the magnitude constraints are tested against $\mathbf{r}_{\text{csd}}$ rather than against x_c . The optimizer thus receives cuts that are derived from a genuinely realizable design, so that a feasible point corresponds to a CSD coefficient vector whose sampled response satisfies the specification. This is the essential difference from quantizing a finished solution.

Cut classification and the retry mechanism. The evaluated candidate is classified as feasible, in which case the best-so-far value is improved and a central cut is taken; as violated, in which case a deep cut is taken from the gradient at $\mathbf{r}_{\text{csd}}$; or as ineffective, in which case the cut does not shrink the ellipsoid. The last case is peculiar to the discrete setting: because the set of admissible points is finite, several consecutive centers can map to the same CSD pattern, producing a cut parallel to an earlier one that removes no new volume. When this occurs the oracle retries, perturbing the quantization decision or re-deriving the pattern from the updated center, for a bounded number of attempts on the order of fifteen. If the retries are exhausted the iteration is abandoned and the ellipsoid is shrunk by the last useful cut. The retry loop is also the mechanism for handling an infeasible discrete subproblem: when no CSD vector satisfies the sampled constraints, the procedure reports the best available continuous point and the discrete feasible set is deemed empty at the current resolution.

Pitfall: the discretization artifact. The oracle certifies feasibility only at the finite sampling grid of $m = c_{\text{disc}}N$ frequencies, where c_{disc} is the sampling factor. Between consecutive samples the squared magnitude can dip below the lower bound even though the design passes at every sample. This is a discretization artifact, the familiar consequence of replacing a semi-infinite constraint by finitely many samples, and it is not an error in the discrete optimization. It is handled by increasing the sampling factor, for example from fifteen to thirty, when the constraints are tight, and by re-checking the final design on a finer grid. The fine-grid check is advisory, because the optimizer never certified that resolution.

4.8.2. Discrete optimization versus quantize-after. The discrete constraint can be treated in two ways, and the contrast exposes why embedding it in the oracle is preferable for tight specifications.

Formulation for the quantize-after approach. The continuous problem

$$\begin{aligned} & \text{minimize} && \max R(\omega) \\ & \text{subject to} && L^2(\omega) \leq R(\omega) \leq U^2(\omega), \quad \forall \omega \in [0, \pi], \\ & && \mathbf{r} \in \mathbb{R}^N, \end{aligned}$$

is solved first, and the discrete constraint is imposed afterwards by

$$\mathbf{h} = S(\mathbf{r}^*), \quad h_{\text{csd}}[k] = \text{csd}(h[k], \text{nnz}).$$

The constraint is absent from the optimization; it acts only on the returned solution.

Formulation for the discrete optimization approach. The admissible set is

restricted to the autocorrelations that are realizable from CSD coefficients,

$$\begin{aligned} & \text{minimize} && \max R_{\text{csd}}(\omega) \\ & \text{subject to} && \overset{\omega}{L^2}(\omega) \leq R_{\text{csd}}(\omega) \leq U^2(\omega), \quad \forall \omega \in [0, \pi], \\ & && \mathbf{r} \in \mathcal{Q}, \end{aligned}$$

where

$$\mathcal{Q} = \{ S^{-1}(\mathbf{h}_{\text{csd}}) \mid h_{\text{csd}}[k] \in \text{CSD}(\text{nmz}) \}$$

and R_{csd} is the squared magnitude of the filter whose coefficients are $\mathbf{h}_{\text{csd}}$. The constraint is enforced inside the oracle, which projects each queried continuous point into $\mathcal{Q}$ before evaluating the sampled constraints.

The two approaches compare as follows.

Aspect	Quantize-after	Discrete optimization
Constraint handling	after optimization	inside the oracle
Search domain	$\mathbb{R}^N$	$\mathcal{Q}$
Returned solution	continuous $\mathbf{r}^*$, then rounded	quantized $\mathbf{r}_{\text{csd}}$
Specification guarantee	none	at the sampled frequencies
Cost per design	single pass	multiple passes with retries
Convergence	faster	slower by roughly a factor of two
Appropriate use	exploration, loose tolerances	production, tight specifications

Why quantize-after fails. At the relaxed optimum the active magnitude constraints are saturated, so the optimum lies on the boundary of the feasible set. The composite map that rounds a coefficient vector and returns its autocorrelation, $S^{-1} \circ \text{csd} \circ S$, is neither a projection onto the feasible set nor exact; it perturbs the autocorrelation by an amount governed by the coefficient quantization step and by the round-trip error of spectral factorization. Because the relaxed optimum is on the boundary, even this perturbation can push part of the passband below L^2 , so the rounded filter can violate the specification substantially rather than marginally. When the transition band is narrow the constraints are necessarily tight, and the naive procedure frequently returns an infeasible design. The relaxed optimum violates the quantization constraint in the precise sense that it was never asked to respect it, and the post hoc rounding is not the projection that would repair the omission.

Choice of approach. Quantize-after is appropriate for quick feasibility studies, for loose tolerances, and as a warm start for the discrete stage, provided the result is verified afterwards. Discrete optimization is appropriate for production integrated circuits and field-programmable gate arrays, where compliance must be guaranteed at the sampling resolution, and for tight passband specifications, where it should be combined with an enlarged sampling factor. A practical workflow may use the first to obtain a starting point and the second to obtain the deliverable.

4.8.3. Example: Multiplierless FIR Filter Design. However, there are still many filter design problems that are non-convex, such as multiplierless FIR filter design problems. Note that in Fig. 4.3, each coefficient associated with a multiplier unit makes the filter power-hungry, especially in *application specific integrated circuits* (ASIC). If each coefficient is quantized and represented as a sum of signed powers of two (SPT), a multiplierless filter can be implemented. Such coefficients are uniquely represented by a canonical signed-digit (CSD) code with a minimum number of non-zero digits [24]. This confines multiplication to additions and shifts. For example, $0.40625 = 13/32 = 2^{-1} - 2^{-3} + 2^{-5}$, so the multiplier can be replaced by three shifters and two adders at much lower cost. However, the coefficient quantization constraint is non-convex, making the convex optimization algorithm not directly applicable. A similar case is the consideration of the finite word-length effect [16].

Attracted by the benefits of this “multiplier-free” approach, many efforts have been devoted to its design techniques. For general problems, integer programming (e.g. [14, 16–18]) can be implemented to achieve the optimal solution. However, its computational cost is prohibitive. Other heuristics, such as genetic algorithms [31] and dynamic-programming methods [5], are likewise inefficient. If the quantization constraint is the only non-convex constraint in the design problem, a lower bound can be efficiently obtained by solving the relaxed problem [10]. Then to make the solution feasible, it can be rounded to the nearest CSD code or used as a starting point of a local search algorithm to obtain a better solution [15]. However, neither method guarantees the feasibility of the final solution. Besides, the local search problem remains non-convex. Therefore, the adopted algorithm may also be inefficient, such as branch-and-bound in [15].

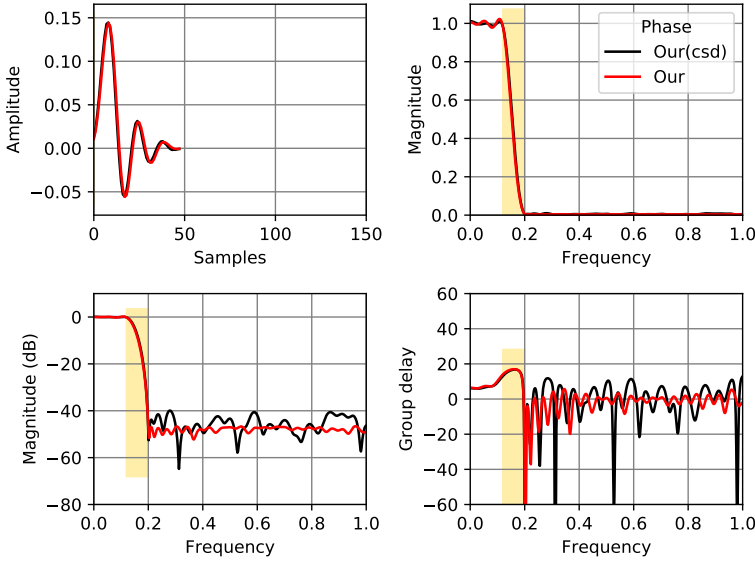


Figure 4.5: Result

4.8.3.1. Verification methodology. Because the quantized design is the deliverable, verification should be conducted on the artifact that the hardware realizes, and at the

resolution at which feasibility was certified.

Verify the exact quantized strings, not the floating-point values. The numerical coefficients returned by the optimizer are produced by spectral factorization and, if the continuous values are inspected, by a further quantization; they therefore carry the round-trip error of S and S^{-1} and can even be quantized twice. A CSD string, by contrast, maps losslessly to a dyadic rational, so reconstructing each coefficient from its string and evaluating the response reproduces exactly what the hardware computes, free of the factorization round trip. A specification check performed on floating-point values can therefore report a failure that the implemented filter does not exhibit, and is a false negative rather than evidence of a defective design.

Use a dual-resolution check. The magnitude response should be evaluated on the oracle grid of $m = c_{\text{disc}}N$ frequencies, which is the resolution at which feasibility was certified and is authoritative for the acceptance decision, and also on a much finer grid as an advisory diagnostic. A dip on the fine grid, occurring between oracle samples, is a discretization artifact rather than a contradiction of the guarantee, and it signals that the sampling factor should be increased. A conservative workflow certifies on the oracle grid, inspects the fine grid, and raises the sampling factor when the fine-grid margin is small. As complementary checks, each quantized coefficient should respect the non-zero digit budget, and the spectral energy of the quantized response should be consistent with $r[0]$.

5. Concluding Remarks. While the ellipsoid method may be perceived as slower than interior-point methods for solving convex problems, it offers distinct advantages, such as the ability to handle problems with a large or infinite number of constraints. Techniques like parallel cuts and efficient implementations have helped to improve the performance of the ellipsoid method, making it a valuable tool in the optimization landscape. Finally, rather than viewing the ellipsoid method as a competitor to other optimization techniques, it should be seen as a companion, with each method offering unique strengths that can be leveraged to solve a wide range of optimization problems effectively.

The advantages above come with limitations that the preceding sections establish and that a prospective user should weigh. The method cannot exploit sparsity in the problem data, so any structural efficiency must be supplied by the separation oracle, and its iteration budget of order n^2 means that interior-point methods scale better as the number of design variables grows (§4.6). The comparison in §4.6 is itself a research-grade implementation set against a mature solver stack: the interior-point path admits warm starts, the stopping tolerances were not harmonized, and the per-iteration overhead of an interpreted environment can reverse the ranking even when the compiled method is faster. Numerical robustness is not automatic either: the direct matrix representation can lose positive definiteness silently, which motivates the factorized variant of §4.1.2; an absolute stopping threshold on a scale-dependent quantity must be paired with the stall guard of §2.2.1; and a triangular factor must be inverted by triangular back-substitution rather than a general symmetric routine (§4.3.3). The oracle frameworks carry approximation risks of their own, including the residual curvature error of affine arithmetic (§3.1.3), the bias introduced when the $2Y$ trust region of the difference-of-convex problem is imposed as a hard constraint (§3.3.4), and the discretization artifacts and retry heuristics of the discrete setting (§4.8). These are the price of a solver that applies where interior-point methods

cannot, and they are the reason its practical viability rests on compiled execution, floating-point discipline, and domain-specific oracle engineering.

One qualification tempers the last of these. The separation oracle is usually the entry barrier, and with capable language-model assistance the authoring of one is no longer the obstacle it once was. Verification is not delegated so easily: the method is only as sound as the oracle it queries, and an invalid cut or a wrong subgradient returns a wrong answer without any signal in the iteration count. The user’s effort shifts from writing the oracle to testing it, a real reduction that nevertheless leaves the domain knowledge, and the responsibility for correctness, where it was.

Appendix A. Cholesky Decomposition and the $LDL^\top$ Witness.

The Cholesky decomposition algorithm is a method used in linear algebra to decompose a Hermitian, positive-definite matrix into the product of a lower triangular matrix and its conjugate transpose.

The Cholesky decomposition of a Hermitian positive-definite matrix A is a unique decomposition where $A = LL^*$, with L being a lower triangular matrix containing real and positive diagonal entries, and L^* representing the conjugate transpose of L . Every real-valued symmetric positive definite matrix and every Hermitian positive definite matrix admits a Cholesky decomposition.

If A is a real matrix that is symmetric and positive-definite, it can be decomposed as $A = LL^T$. Here, L represents a real lower triangular matrix with positive diagonal entries.

The Cholesky and $LDL^\top$ decompositions are matrix decomposition methods utilized in linear algebra for disparate purposes, exhibiting distinctive properties.

The Cholesky decomposition is a method for decomposing a Hermitian, positive-definite matrix into the product of a lower triangular matrix and its conjugate transpose. The Cholesky decomposition is typically a faster and more numerically stable method than the $LDL^\top$ decomposition. Nevertheless, the input matrix must be positive definite for this to be effective.

The $LDL^\top$ decomposition factors a symmetric matrix into a unit lower triangular matrix, a diagonal matrix, and the transpose of the lower triangular matrix. Because it avoids the square roots of Cholesky, it can be faster, and when a symmetric indefinite factorization with 2×2 pivots is used it also handles matrices that are not positive definite.

$$\begin{aligned} \mathbf{A} = \mathbf{LDL}^\top &= \begin{pmatrix} 1 & 0 & 0 \\ L_{21} & 1 & 0 \\ L_{31} & L_{32} & 1 \end{pmatrix} \begin{pmatrix} D_1 & 0 & 0 \\ 0 & D_2 & 0 \\ 0 & 0 & D_3 \end{pmatrix} \begin{pmatrix} 1 & L_{21} & L_{31} \\ 0 & 1 & L_{32} \\ 0 & 0 & 1 \end{pmatrix} \\ &= \begin{pmatrix} D_1 & & \\ L_{21}D_1 & L_{21}^2D_1 + D_2 & \\ L_{31}D_1 & L_{31}L_{21}D_1 + L_{32}D_2 & L_{31}^2D_1 + L_{32}^2D_2 + D_3 \end{pmatrix}. \end{aligned}$$

(symmetric)

If A is real, the following recursive relations apply for the entries of D and L :

$$D_j = A_{jj} - \sum_{k=1}^{j-1} L_{jk} L_{jk}^* D_k,$$

$$L_{ij} = \frac{1}{D_j} \left(A_{ij} - \sum_{k=1}^{j-1} L_{ik} L_{jk}^* D_k \right) \quad \text{for } i > j.$$

Once more, the pattern of access enables the entire computation to be performed in-place.

The Cholesky or $LDL^\top$ decomposition can be computed using either row-based or column-based methods:

- Column-Based: In this approach, the computation is conducted in a column-wise manner. The inner loops calculate the current column using a matrix-vector product that accumulates the effects of previous columns.
- Row-Based: In this approach, the calculations are conducted row by row. The inner loops are responsible for computing the current row, which is achieved by solving a triangular system involving previous rows.

The selection of each outer loop index results in a unique Cholesky algorithm, named after the portion of the matrix updated by the fundamental operation within the inner loops. The choice between a row-based or column-based method depends on the specific requirements of the problem, as well as the system properties, such as memory layout and access patterns. The row-based decomposition with lazy evaluation allows the cutting-plane construction to be completed in $O(p^3)$. This allows for the effective implementation of oracles.

The Cholesky decomposition provides a witness vector that certifies a matrix is not positive definite. If a matrix fails the Cholesky decomposition, it is not positive definite. During the decomposition process, the diagonal of the lower triangular matrix should be calculated by finding the square root of a value, denoted as x . If x is less than zero, this indicates that the matrix is not positive definite. This failure serves as evidence that the matrix in question is not positive definite.

In the event that the Cholesky decomposition is unsuccessful due to a negative diagonal element, this indicates that the leading principal submatrix up to that point is not positive definite. The confirming vector is a standard basis vector with a 1 in the position of the failed diagonal element and zeros elsewhere; sandwiching it between the original matrix and its transpose yields a negative value, which certifies that the matrix is not positive definite.

The oracle should perform a *row-based* Cholesky decomposition such that $F(x_0) = R^\top R$. The notation $A_{:,p,:}$ is used to denote a submatrix $A(1:p, 1:p) \in \mathbb{R}^{p \times p}$. If the Cholesky decomposition fails at row p , there exists a vector e_p , defined as $(0, 0, \dots, 0, 1)^\top \in \mathbb{R}^p$. This can be expressed as follows:

- $v = R_{:,p,:}^{-1} e_p$, and
- $v^\top F_{:,p,:}(x_0) v < 0$.

Algorithm 4 Row-based Cholesky witness for an LMI oracle

Require: symmetric $F(x_0) \in \mathbb{R}^{n \times n}$

Ensure: “PD”, or a witness v with $v^\top F v < 0$

```
1: for  $i = 1$  to  $n$  do
2:   for  $j = 1$  to  $i$  do
3:      $d \leftarrow F_{ij} - \sum_{k < j} L_{ik} L_{jk} D_k$ 
4:     if  $i = j$  then
5:        $D_i \leftarrow d$ 
6:     else
7:        $L_{ij} \leftarrow d / D_j$ 
8:     end if
9:   end for
10:  if  $D_i \leq 0$  then
11:     $p \leftarrow i$ ;  $v \leftarrow R_{:,p}^{-1} e_p$ 
12:    return “not PD”,  $v$ 
13:  end if
14: end for
15: return “PD”
```

The cut (g, β) is then given by the following equation:

$$(-v^\top \partial F_{:,p}(x_0) v, -v^\top F_{:,p}(x_0) v).$$

References.

- [1] Adams, J. W., and Sullivan, J. L., Peak-constrained least-squares optimization, *Signal Processing, IEEE Transactions on* **46** (1998) 306–321.
- [2] Aliabadi, Hossein, and Salahi, Maziar, Robust geometric programming approach to profit maximization with interval uncertainty, *Computer Science Journal of Moldova* **21** no. 1 (2013) 86–96.
- [3] Bland, R. G., Goldfarb, D., and Todd, M. J., The ellipsoid method: A survey, *Operations Research* **29** no. 6 (1981) 1039–1091.
- [4] Boyd, Stephen, and Vandenberghe, Lieven, *Convex optimization*, Cambridge university press, 2009.
- [5] Chen, Chao-Liang, and Willson Jr, Alan N, A trellis search algorithm for the design of FIR filters with signed-powers-of-two coefficients, *Circuits and Systems II: Analog and Digital Signal Processing, IEEE Transactions on* **46** no. 1 (1999) 29–39.
- [6] Cherkassky, Boris V, and Goldberg, Andrew V, Negative-cycle detection algorithms, *Mathematical Programming* **85** no. 2 (1999) 277–311.
- [7] Dasdan, Ali, Experimental analysis of the fastest optimum cycle ratio and mean algorithms, *ACM Transactions on Design Automation of Electronic Systems (TODAES)* **9** no. 4 (2004) 385–418.
- [8] Dasdan, Ali, Experimental analysis of the fastest optimum cycle ratio and mean algorithms, *ACM Transactions on Design Automation of Electronic Systems* **9** no. 4 (2004) 385–418.

- [9] Dasdan, Ali, and Gupta, Rajesh K, Faster maximum and minimum mean cycle algorithms for system-performance analysis, *IEEE transactions on computer-aided design of integrated circuits and systems* **17** no. 10 (1998) 889–899.
- [10] Davidson, Timothy N, Enriching the art of FIR filter design via convex optimization, *Signal Processing Magazine, IEEE* **27** no. 3 (2010) 89–101.
- [11] Diggle, P. J., and Jr., P. J. Ribeiro, *Model-based Geostatistics*, Springer, New York, 2007.
- [12] Goodman, T. N. T., Micchelli, C. A., Rodriguez, G., and S. Seatzu, and, Spectral factorization of laurent polynomials, *Advances Comput. Math.* **7** no. 4 (1997) 429–454.
- [13] Khachiyan, L. G., A polynomial algorithm in linear programming, *Soviet Mathematics Doklady* **20** (1979) 191–194.
- [14] Kodek, Dusan M, Design of optimal finite wordlength FIR digital filters using integer programming techniques, *Acoustics, Speech and Signal Processing, IEEE Transactions on* **28** no. 3 (1980) 304–308.
- [15] Kodek, Dusan, and Steiglitz, Kenneth, Comparison of optimal and local search methods for designing finite wordlength FIR digital filters, *Circuits and Systems, IEEE Transactions on* **28** no. 1 (1981) 28–32.
- [16] Lim, Yong Ching, Parker, S, and Constantinides, AG, Finite word length FIR filter design using integer programming over a discrete coefficient space, *Acoustics, Speech and Signal Processing, IEEE Transactions on* **30** no. 4 (1982) 661–664.
- [17] Lim, Yong Ching, Yang, Rui, Li, Dongning, and Song, Jianjian, Signed power-of-two term allocation scheme for the design of digital filters, *Circuits and Systems II: Analog and Digital Signal Processing, IEEE Transactions on* **46** no. 5 (1999) 577–584.
- [18] Lim, Yong, and Parker, S, FIR filter design over a discrete powers-of-two coefficient space, *Acoustics, Speech and Signal Processing, IEEE Transactions on* **31** no. 3 (1983) 583–591.
- [19] Liu, Xuexin, Luk, Wai-Shing, Song, Yu, Tang, Pushan, and Zeng, Xuan, Robust analog circuit sizing using ellipsoid method and affine arithmetic, in *Proceedings of the 2007 asia and south pacific design automation conference*, IEEE Computer Society, pp. 203–208.
- [20] Mitra, Sanjit Kumar, and Kuo, Yonghong, *Digital signal processing: A computer-based approach*, vol. 2, McGraw-Hill New York, 2006.
- [21] Oppenheim, Alan V, Schafer, Ronald W, Buck, John R, and others, *Discrete-time signal processing*, vol. 2, Prentice hall Englewood Cliffs, NJ, 1989.
- [22] Orlin, James B, and Rothblum, Uriel G, Computing optimal scalings by parametric network algorithms, *Mathematical programming* **32** no. 1 (1985) 1–10.
- [23] Parks, T. W., and McClellan, J. H., Chebyshev approximation for nonrecursive digital filters with linear phase, *Circuit Theory, IEEE Transactions on* **CT-19** no. 2 (1972) 189–194.
- [24] Reitwiesner, George W., Binary Arithmetic, *Advances in Computers* **1** (1960) 231–308.
- [25] Schabenberger, O., and Gotway, C. A., *Statistical Methods for Spatial Data Analysis*, Chapman & Hall/CRC, Florida, 2005.

- [26] Selesnick, I. W., Lang, M., and Burrus, C. S., Constrained least square design for FIR filters without specified transition bands, *Signal Processing, IEEE Transactions on* **44** (1996) 1879–1892.
- [27] Shor, N. Z., Cut-off method with space extension in convex programming problems, *Cybernetics* **13** no. 1 (1977) 94–96.
- [28] Steiglitz, K., Parks, T. W., and Kaiser, J. F., METEOR: A constraint-based FIR filter design program, *Signal Processing, IEEE Transactions on* **40** (1992) 1901–1909.
- [29] Tarjan, R.E., *Shortest paths*, AT&T Bell Laboratories, 1981.
- [30] Wu, Shao-Po, Boyd, Stephen, and Vandenberghe, Lieven, FIR filter design via spectral factorization and convex optimization, in *Applied and computational control, signals, and circuits*, Springer, pp. 215–245.
- [31] Xu, Dexiang J, and Daley, Michael L, Design of optimal digital filter using a parallel genetic algorithm, *Circuits and Systems II: Analog and Digital Signal Processing, IEEE Transactions on* **42** no. 10 (1995) 673–675.
- [32] Yudin, D. B., and Nemirovskii, A. S., Informational complexity and effective methods of solution for convex extremal problems, *Ekonomika i Matematicheskie Metody* **12** no. 2 (1976) 357–369.
- [33] Zhou, Xingbao, Luk, Wai-Shing, Zhou, Hai, Yang, Fan, Yan, Changhao, and Zeng, Xuan, Multi-parameter clock skew scheduling, *Integration, the VLSI Journal* **48** (2015) 129–137.